

# **An Explainable AI Pipeline for Academic Integrity Verification**

*Transparent AI-Text Detection with Argument-Aware Question Generation*

Mykhailo Shpyl

M.Sc. in Artificial Intelligence and Big Data Analytics, 2026



Department of Computing, ATU Donegal, Port Road, Letterkenny, Co. Donegal, Ireland.

*Working draft for supervisor review, June 2026*

# **An Explainable AI Pipeline for Academic Integrity Verification**

*Transparent AI-Text Detection with Argument-Aware Question Generation*

Author: Mykhailo Shpyl

Supervised by: Dr. Vini Vijayan

A thesis submitted in partial fulfilment of the requirements for the

**Master of Science in Artificial Intelligence and Big Data Analytics**

Submitted to Atlantic Technological University

*Arna chur isteach chuig Ollscoil Teicneolaíochta an Atlantaigh*

August 2026

## Declaration

I hereby certify that the material, which I now submit for assessment on the programmes of study leading to the award of Master of Science in Artificial Intelligence and Big Data Analytics, is entirely my own work and has not been taken from the work of others except to the extent that such work has been cited and acknowledged within the text of my own work. No portion of the work contained in this thesis has been submitted in support of an application for another degree or qualification to this or any other institution. I understand that it is my responsibility to ensure that I have adhered to ATU's rules and regulations.

I hereby certify that the material on which I have relied on for the purpose of my assessment is not deemed as personal data under the GDPR Regulations. Personal data is any data from living people that can be identified. Any personal data used for the purpose of my assessment has been pseudonymised and the data set and identifiers are not held by ATU. Alternatively, personal data has been anonymised in line with the Data Protection Commissioner's Guidelines on Anonymisation. All datasets used in this work are publicly available and openly licensed, and no human participants were involved in the study.

I consent that my work will be held for the purposes of education assistance to future students and will be shared on the ATU Donegal (Computing) website ([atucomputingdonegal.com](http://atucomputingdonegal.com)) and Research THEA website (<https://research.thea.ie/>). I understand that documents once uploaded onto the website can be viewed throughout the world and not just in Ireland. Consent can be withdrawn for the publishing of material online by emailing Jade Lyons, Head of Department, at [Jade.Lyons@atu.ie](mailto:Jade.Lyons@atu.ie) to remove items from the ATU Donegal Computing website, and by emailing Denise McCaul, Systems Librarian, at [denise.mccaul@atu.ie](mailto:denise.mccaul@atu.ie) to remove items from the Research THEA website. Material will continue to appear in printed formats once published, and as websites are a public medium, ATU cannot guarantee that the material has not been saved or downloaded.

Signature of Candidate: Mykhailo Shpyl

Date: \_\_\_\_\_

## Acknowledgements

I would like to thank my supervisor, Dr. Vini Vijayan, for her guidance and feedback throughout this project, and the staff of the MSc in Artificial Intelligence and Big Data Analytics at Atlantic Technological University for their support.

## Abstract

Universities increasingly rely on automatic detectors to judge whether student work was written with generative AI. Most detectors return a single percentage with nothing behind it, which a lecturer cannot defend if a student challenges it, and which says nothing about whether the student understands the work they submitted. This dissertation develops an explainable pipeline for academic integrity verification. It detects likely AI-generated text, explains which features drove each decision, extracts the claims in a flagged submission, and generates verification questions tied to those claims so a lecturer can check understanding in a short conversation. The detector was trained on a balanced corpus of 640 human essays from the British Academic Written English corpus and 640 length-matched and topic-matched AI essays generated locally with Llama 3.1. An initial model reached a perfect score, which was treated as a warning rather than a result: a reproducible audit found the corpus leaked formatting markup, removed it from both classes, and confirmed that the classes remain separable on writing style alone (a function-words-only model still reaches 99.5 percent; the cleaned transformer scores an F1 of 0.99). The decisions are then explained, and the explanations tested: token-level attributions on the transformer prove diffuse and only weakly faithful, while SHAP over stylometric features passes an ablation-based faithfulness test and gives the lecturer-facing explanation. On an external benchmark the detector transfers to unseen generators (F1 0.97) but degrades across domains (F1 0.79), where its errors concentrate on falsely flagging human academic text, which quantifies the fairness risk that motivates the project. A first slice of the question-generation pipeline turns a flagged essay into claim-grounded verification questions with sentence-level provenance, and a judge-free discrimination simulation evaluates them, showing that questions forcing a student to reproduce their own reasoning discriminate better than questions answerable from general knowledge. A commercial-versus-local comparison over fourteen essays finds a small but statistically significant commercial advantage (paired difference 0.036,  $p = 0.040$ ) with the local open model competitive before any fine-tuning, and a first LLM-as-judge run whose near-ceiling ratings show no correlation with the objective measure, supporting the decision to anchor judges to the simulation. A QLoRA fine-tune of a small local model on an educational-question corpus appeared on the same simulation to beat the commercial model, but inspection of its output showed it had collapsed into degenerate multiple-choice stems

that game the metric: a negative result that redirects the local-model work toward open-ended training data and shows the discrimination measure needs a well-formedness guard. Re-fine-tuning on open-ended questions (SQuAD) confirmed the diagnosis, removing the degeneracy entirely and yielding a real but modest gain over the base model (0.10 vs 0.03,  $p = 0.0003$ ) on well-formed questions, still short of a strong verification-question generator. This document is a working progress draft for supervisor review and will be revised further before final submission.

## Acronyms

| Acronym | Definition                                       |
|---------|--------------------------------------------------|
| AI      | Artificial Intelligence                          |
| NLP     | Natural Language Processing                      |
| LLM     | Large Language Model                             |
| GenAI   | Generative Artificial Intelligence               |
| BAWE    | British Academic Written English (corpus)        |
| TTR     | Type-Token Ratio                                 |
| POS     | Part of Speech                                   |
| BIO     | Begin-Inside-Outside (sequence labelling scheme) |
| QLoRA   | Quantised Low-Rank Adaptation                    |
| SHAP    | SHapley Additive exPlanations                    |
| IG      | Integrated Gradients                             |
| FPR     | False-Positive Rate                              |
| HPC     | High-Performance Computing                       |

# Table of Contents

*Update this field in Word (select all, then press F9) to populate page numbers.*



## Table of Figures

- Figure 3.1** What each detection signal achieves on its own
- Figure 3.2** The words the cleaned-text model keys on (style, not topic)
- Figure 3.3** DeBERTa on the held-out test set after markup removal
- Figure 3.4** Essays in function-word style space: two clusters
- Figure 3.5** Detector F1 with 95% bootstrap confidence intervals
- Figure 5.1** Integrated Gradients token attributions for a matched essay pair
- Figure 5.2** Faithfulness by ablation: the signal is diffuse
- Figure 5.3** SHAP on the stylometric detector (feature-level explanation)
- Figure 6.1** Transfer to unseen generators on essays
- Figure 6.2** In-domain vs cross-generator vs cross-domain F1
- Figure 6.3** Cross-domain failure modes by domain
- Figure 7.1** Bloom's-level classification: trained BERT vs the keyword heuristic
- Figure 7.2** Argument-component extraction: strict span F1 on Persuasive Essays
- Figure 7.3** QLoRA fine-tune vs base Qwen 3B: discrimination on the same claims
- Figure 8.1** Discrimination simulation: claim-grounded vs generic questions
- Figure 8.2** Commercial vs local question generation, balanced across essays
- Figure 8.3** Four question writers on one fixed claim set (like-for-like)
- Figure 8.4** Question-quality audit: the fine-tune's discrimination score is an artifact
- Figure 8.5** Fixing the fine-tune: v2 on open-ended data (real but modest gain)

# Chapter 1: Introduction

## 1.1 Overview

This dissertation builds an explainable pipeline that helps a lecturer check the integrity of a student submission without having to treat a detector's score as the final word. It does two connected things. First it decides whether a piece of writing is likely AI-generated, and rather than returning a bare percentage it shows which features of the writing drove that decision. Second, when a submission is flagged, it reads the student's own argument, pulls out the specific claims they made, and turns those claims into verification questions a lecturer can ask in a short conversation. The aim is not to catch students out. It is to give the lecturer transparent, defensible evidence, tied to the student's own words, that a fair decision can rest on.

Two design choices run through the whole system. Every stage is meant to be explainable, so a lecturer can see and justify why a submission was flagged and where each question came from, instead of pointing at a number they cannot account for. And the question generation is built twice, once with a commercial large language model and once with a smaller open model that runs on a single laptop, so the project can ask a question that matters to any institution: how close can a locally run, low-cost model get to an expensive commercial one for this task (Oketch et al., 2025). That comparison is a central contribution.

The intended user is a lecturer on a large module, somewhere between 150 and 300 students, who has neither the time to prepare a fair set of verification questions for every flagged essay nor a good way to defend a black-box score if a student challenges it. The system is aimed squarely at that person. It is designed to save time, to treat students fairly, including students who do not write in English as a first language and whom existing detectors tend to penalise (Liang et al., 2023), and to produce something a lecturer can stand over.

## 1.2 Background and context

Over the last few years generative AI tools have moved from a curiosity to something most students have used at least once (Jin et al., 2024). Drafting an opening paragraph, rephrasing an awkward sentence, or getting a hard idea explained is now a few seconds of typing into a chatbot. That is genuinely useful for learning, but it has also put pressure on a basic academic question: is this piece of work the student's own, and does it reflect their own understanding?

Universities responded quickly, and a market of AI-text detectors appeared almost as fast as the writing tools did (Wu et al., 2025). Broadly these detectors fall into two families. Some are zero-shot or statistical: they use a language model's own probabilities to judge how predictable a passage is, on the assumption that machine text sits in smoother, more probable regions than human text (Mitchell et al., 2023). Others are supervised: a transformer is fine-tuned on labelled human and AI examples and then classifies new text (He et al., 2021). Commercial services usually combine such methods behind an interface and return a single figure, an AI-likelihood percentage.

The trouble is not that these tools never work. It is that they were built fast, they are opaque, and they are fragile in the situations that matter most. Their accuracy drops sharply on text they were not trained for, and they can be evaded by simple paraphrasing (Krishna et al., 2023). More seriously for a university, they hand back a number with nothing behind it, and there is growing evidence that the errors are not spread evenly: writing by non-native English speakers is flagged as AI far more often than it should be (Liang et al., 2023). A tool that is confident, unexplained, and unfair is a poor basis for an academic-integrity decision.

### **1.3 Problem statement**

The core problem I keep returning to is that current detectors give a lecturer a percentage and stop there. Someone who sees "82 percent AI" has no way to explain that number if the student pushes back, and no real way to know whether it is even right for this particular essay. That is a weak and risky position to accuse a person from, and the cost of being wrong falls on the student, whose record and standing are at stake. A score with no reasons behind it cannot carry that weight.

There is a fairness problem on top of the opacity. Because the errors are biased, the students most likely to be wrongly flagged are often those who are already disadvantaged, including people writing in a second language (Liang et al., 2023). A system that quietly repeats that bias, and cannot even show its reasoning, is worse than no system at all.

The deeper problem is that even a correct flag does not answer the question a lecturer actually cares about. Knowing that text is probably AI-generated does not tell you whether the student understands the material they handed in. One student might have used a tool heavily and still grasp the argument; another might have written every word themselves and

understood little. A flag is a suspicion about how a text was produced, not evidence about what the student knows. What a lecturer needs, and what no detector provides, is a fair way to move from that suspicion to a check of understanding.

Closing that distance is the problem this project takes on: make the detection transparent and defensible, and then turn a flag into specific, source-grounded questions that test whether the student can account for their own work.

## **1.4 Motivation**

I am aiming this at the lecturer who runs a large module and has no time to sit with each flagged essay and work out a fair way to check it. That person needs something they can defend, that treats students fairly, and that saves them time rather than adding to the pile. The fairness side matters to me in particular, because the research already shows these detectors can be biased against people who do not write in English as a first language (Liang et al., 2023), and I do not want to build something that quietly repeats that. A system that is transparent about its reasoning is also a system whose mistakes can be seen and argued with, which is exactly what a fair process needs.

## **1.5 The verification gap**

So there is a gap. On one side is detection, which is improving but stays a black box. On the other side is the question that actually matters: does this student understand their own submission? Nothing I found connects the two. Nothing takes a flag and turns it into specific questions, drawn from the student's own claims, that a lecturer can ask in a short conversation to see whether the understanding is there. That gap is what this project tries to fill.

## **1.6 Research question and objectives**

The main question I am asking is this: how can an explainable AI pipeline be designed to support lecturers in academic integrity verification, by combining transparent AI text detection with argument-aware question generation, and how far can locally fine-tuned open-source models match commercial ones at producing interpretable, defensible outputs for this task.

To get there the objectives are roughly:

- detect AI-written text in a way that can be explained, not just scored;
- show which words and features drove each decision, and test that those explanations are faithful;
- pull out the claims and evidence in a flagged essay and tie each one to its source;
- generate verification questions from those claims, and compare a commercial model against a local open-source one;
- label each question by cognitive level as a quality check; and
- evaluate the whole thing with objective measures rather than opinion.

## 1.7 Scope and boundaries

To keep this doable in the time I have, the scope is deliberately tight. The detector works with two classes only, human and AI. I dropped the earlier idea of partial-AI classes because they overlap too much to separate reliably; that may come back later as future work if the two-class version works well. The pipeline has six parts (detection, explainability, argument mining, question generation, a Bloom's level check, and the output guide), and the datasets are fixed in advance. There are no human participants in the study, which keeps it clear of needing ethics approval and keeps the focus on the system itself.

## 1.8 Contributions

What I think this project adds:

- a pipeline that goes from a detection flag to transparent, defensible evidence, instead of stopping at a score;
- verification questions that are tied back to the student's own writing, so a lecturer can see where each one came from;
- a direct comparison of a locally run open model against a commercial one for this task, which matters for any institution thinking about cost and control;
- an objective way to measure whether a question really tests understanding; and
- an honest look at how the detector treats native versus non-native writers.

## 1.9 Dissertation outline

The rest of the document is laid out as follows.

Chapter 2 reviews the recent literature the project draws on: AI-text detection and its benchmarks, stylometric features, explainability and faithfulness, argument mining, question generation and its evaluation, and the fairness evidence that motivates the design.

Chapter 3 sets out the detection methodology and the first results: the dataset, how the matched AI essays were built, and the audit that found and removed a corpus artefact behind an initially perfect score.

Chapter 4 describes the implementation of the components built so far, with the engineering decisions that the 8 GB laptop budget forced.

Chapter 5 explains the detector's decisions and tests whether those explanations are faithful, comparing token-level attributions on the transformer against SHAP over the stylometric features.

Chapter 6 tests robustness: how the detector transfers to generators it never saw and to other kinds of text, and what its failures mean for fairness.

Chapter 7 presents the first slice of the core contribution: turning a flag into verification questions drawn from the student's own claims, with sentence-level provenance and a Bloom's level on each question.

Chapter 8 evaluates those questions with a judge-free discrimination simulation and reports the first scaled comparison between the commercial and local backends.

The chapters that follow in the final document cover the remaining components (the trained argument miner and the Bloom's classifier), the fine-tuned local backend and the completed commercial-versus-local comparison, the assembled Verification Interview Guide, a discussion of what the results mean and where the limits are, and the conclusion with future work.

## Chapter 2: Literature Review

### 2.1 Scope of the review

This review covers the parts of the literature the project draws on and argues against: detecting AI-generated text, the stylometric features behind the detector, explainability and how to test whether an explanation is honest, argument mining, question generation, how question quality is judged, and fairness. The sections follow the pipeline so the review builds toward the gap the project fills. I anchored the search on the specific methods and datasets already chosen (the DeBERTa detector, the M4 benchmark, the Persuasive Essays corpus, Bloom's taxonomy, the answerability idea behind the evaluation) and then branched out to alternatives and criticisms.

### 2.2 Detecting AI-generated text

Work on detecting machine-generated text splits into two broad families, and a recent survey by Wu et al. (2025) organises the field along these lines, adding watermarking and human-assisted methods.

The first family is zero-shot and statistical. These methods use a language model's own probabilities rather than a trained classifier. The clearest example is DetectGPT (Mitchell et al., 2023), which observes that machine-generated passages tend to occupy negative-curvature regions of a model's log-probability surface, and turns that observation into a curvature test that needs no labelled data. The appeal is that nothing has to be trained; the weakness is that the signal is subtle and can be disturbed.

The second family is supervised. A pretrained transformer is fine-tuned on labelled human and AI examples and then classifies new text. The transformer this project uses, DeBERTa (He et al., 2021), improves on earlier encoders with a disentangled attention mechanism and was chosen as the primary detector for that reason, with RoBERTa (Liu et al., 2019) as a comparison. Supervised detectors are accurate in-domain but depend heavily on what they were trained on.

Progress is measured on shared benchmarks. The M4 corpus (Wang et al., 2023) collects human and machine text across several generators, domains, and languages, and the SemEval-2024 Task 8 shared task built on it (Wang et al., 2024). I use that benchmark to test

whether a detector trained on one generator and one kind of text transfers to unseen generators and unseen domains.

Two weaknesses matter for this project. First, detectors are brittle: Krishna et al. (2023) show that a strong paraphraser drops DetectGPT's accuracy from 70.3% to 4.6% without changing the meaning, and accuracy also falls under domain shift. Second, and more important here, commercial detectors return a single opaque score. A number with no reasons behind it is a weak basis for an academic-integrity decision, and that black-box problem is the starting point for this project.

## 2.3 Stylometric features

Alongside the transformer, the detector uses stylometric features, which describe how a text is written rather than what it is about: perplexity or predictability under a language model, burstiness (variation in sentence length and structure), type-token ratio, sentence-length variance, and part-of-speech distributions. Each is meant to capture a way in which fluent machine text tends to differ from human writing, being smoother and less varied.

There is direct evidence that these surface properties carry a real, and sometimes harmful, signal. Liang et al. (2023) attribute the misclassification of non-native English writing to its lower perplexity, that is, its more limited linguistic variability, which makes it look more machine-like to a perplexity-based detector. That finding is a double lesson: stylometric features do separate the classes, but they can also encode bias, so the pipeline treats them as interpretable evidence whose behaviour must itself be checked. Recent work confirms that hand-crafted features (perplexity, readability, error-based and lexical features) can classify AI-generated and even AI-rephrased text, alone and alongside neural models (Mindner et al., 2023), which supports the hybrid design used here.

## 2.4 Explainability and faithfulness

The explainability layer is the point of the project, so this section covers both the attribution methods and, just as important, how to check whether an explanation is honest.

Two attribution methods are used. Integrated Gradients (Sundararajan et al., 2017) attributes a prediction to its input features while satisfying two axioms, Sensitivity and Implementation Invariance, and is applied here to the transformer's tokens. SHAP (Lundberg and Lee, 2017)



assigns each feature a Shapley-value importance and is applied to the stylometric features, where it gives a lecturer-facing, feature-level explanation. Attention weights are sometimes offered as explanations too, but that use has been criticised (Jain and Wallace, 2019), which is one reason this project does not rely on attention.

An explanation is only useful if it is faithful, meaning the things it highlights are actually what drove the decision. DeYoung et al. (2020) introduced the ERASER benchmark and the comprehensiveness and sufficiency metrics, which measure faithfulness by removing or keeping the features an explanation calls important and seeing whether the prediction moves. This project uses exactly that kind of ablation test, and it is what showed the transformer's token-level attributions to be diffuse and only weakly faithful, while the stylometric SHAP explanation held up.

## **2.5 Argument mining**

To turn a flag into questions grounded in the student's own argument, the pipeline has to extract the claims and evidence in an essay. The foundational resource is the Persuasive Essays corpus of Stab and Gurevych (2017), which annotates argument components (claims, premises) and the relations between them, and frames the task as token-level sequence labelling plus relation classification. The argument-mining component here follows that setup; more recent work applies transformer ensembles, and even LLM refinement, to argument component classification across several corpora (Pietron et al., 2024).

Provenance is the reason argument mining matters here rather than mere extraction. Each extracted claim must point back to the exact source passage it came from, so that a generated question can be tied to where it originated and a lecturer can defend it. Extraction without provenance would reintroduce the black-box problem the project is trying to remove.

## **2.6 Automatic question generation**

Question generation has moved from neural models that generate a question from a passage to large language models prompted to do the same, with growing interest in controllable and grounded generation (Guo et al., 2024). For verification, grounding is the crucial property: a question is only defensible if it can be traced to a specific claim in the student's own text, so the generator works per extracted claim and records the source sentences.

The comparison at the centre of this project, a commercial model against a locally run open model, sits in the wider question of cost and control in LLM deployment. In the closest study to this setting, Oketch et al. (2025) compared closed and open LLMs for automated essay scoring and found open models such as Llama 3 and Qwen2.5 comparable to GPT-4 in performance and fairness at up to 37 times lower cost. An institution that could run verification on a laptop rather than a paid API would have a cheaper and more controllable option, if the quality holds, which is the empirical question the project tests.

## **2.7 Bloom's taxonomy and cognitive level**

Questions are labelled by cognitive level as a quality control, using the revised Bloom's taxonomy of Anderson and Krathwohl (2001), which orders cognitive processes from Remember and Understand through Apply and Analyse to Evaluate and Create. The taxonomy gives a principled way to describe what a question demands of a student.

For the educational side there is a directly relevant dataset. EduQG (Hadifar et al., 2022) provides 3,397 questions with their source documents, of which 903 are annotated with a Bloom's cognitive level, which makes it suitable for training and checking the Bloom's classifier used here. Automatic classification of question cognitive level is an active research task, where approaches from classical classifiers through transformers to large language models have been applied (Kumar et al., 2025).

## **2.8 Evaluating generated questions**

The project's main evaluation asks whether a question genuinely tests understanding, and the closest prior idea is answerability-based, reference-free evaluation. RQUGE (Mohammadshahi et al., 2023) scores a generated question not by comparing it to a reference question but by whether a question-answering module can answer it from the given context. The discrimination simulation used in this project is in the same spirit: it asks whether a model can answer a question with the source versus without it, and treats the gap as the question's discriminative value.

The supplementary evaluation uses an LLM-as-judge, which is increasingly common but has known reliability and bias problems, including position, verbosity, and self-enhancement biases, so its output has to be checked against something independent before it counts

(Zheng et al., 2023). Cross-model agreement is quantified with Krippendorff's alpha (Hayes and Krippendorff, 2007), and anchored to the objective discrimination results.

## **2.9 Fairness and bias in detection**

Fairness is not a side issue for this project; it is one of its motivations. Liang et al. (2023) tested seven widely used GPT detectors on TOEFL essays by non-native English writers and found an average false-positive rate of 61.3%, against about 5.1% for essays by native writers. In other words, the detectors systematically misclassified competent non-native writing as AI-generated. Chapter 6 measures the same failure mode directly on my own detector and treats it as evidence, not a footnote, which is why transparency and defensibility run through the design.

## **2.10 Summary and the gap**

Detection is improving but stays opaque and, on the evidence above, sometimes unfair. Explainability methods exist and can be tested for faithfulness. Argument mining can recover a student's claims with provenance. Question generation and its evaluation are maturing, and Bloom's taxonomy gives a way to grade cognitive level. What is missing is anything that joins these up: nothing takes a detection flag and turns it into transparent, source-grounded verification questions that let a lecturer fairly check whether a student understands their own work. That gap, and the practical question of whether a local model can do the job as well as a commercial one, is what this project addresses.

## Chapter 3: AI text detection (methodology and first results)

### 3.1 What this component has to do

The first stage of the pipeline decides whether a piece of submitted writing is the student's own or was produced by a generative model. Everything downstream depends on it, but the score on its own is not the point of the project. The point is a decision a lecturer can defend. So I am as interested in why the model decides what it decides as in how often it is right.

### 3.2 The detection corpus

I needed text that is labelled human or AI and that does not give the answer away for the wrong reason. The human side is a stratified sample of 640 real student essays from BAWE (the sampling is described in the sample design note). For the AI side I generated one essay per human essay with Llama 3.1 8B running locally, matched on topic, keywords and target length. Length matching matters: if the AI essays were systematically longer or shorter, a detector could score well just by counting words and would learn nothing about style. After generation the lengths line up closely (Pearson  $r$  about 0.98 between each AI essay and its human source, mean length ratio about 1.05), so length is not the signal.

That gives 1,280 essays, 640 human and 640 AI. I split them at the level of the student, not the essay, so no student appears in both training and test. If the same person's writing sat on both sides of the split, the model could memorise that person rather than learn the general difference. The split is 438 / 102 / 100 pairs for train / validation / test.

### 3.3 The detector

The detector is a fine-tuned transformer. DeBERTa-v3-base is the primary model and RoBERTa-base is a comparison, both fine-tuned to classify human (0) against AI (1) with the same settings and the same student-level splits. I report accuracy, precision, recall, F1 and the confusion matrix, plus a separate false-positive rate for native and non-native English writers, because wrongly flagging a non-native student as AI is the failure mode I most want to avoid. A stylometric feature model (perplexity, burstiness, vocabulary richness, part-of-speech mix) is being added as the second half of the hybrid; this chapter is the transformer half on its own.

### 3.4 The result, and why I did not trust it at first

The first trained detector scored a perfect F1 of 1.00 on the held-out test set, for both DeBERTa and RoBERTa. A perfect score is not a result to celebrate, it is a result to investigate. In a project that is about making detection defensible, presenting a number I could not explain would be the opposite of the point. So before using it I ran an audit (`src/detection/audit_detector.py`) to find out whether the score was real or a shortcut.

### 3.5 The audit

The audit asked four questions.

**Is the test set contaminated?** No. I checked that no student sits in more than one split, that each human and AI pair stays in the same split, and that no exact-duplicate text crosses from training into test. All three came back clean across 394 students. So the perfect score is not the model having seen the answers.

**Is there a trivial giveaway in the text?** Yes, and this is the important finding. The human essays come from the BAWE plain-text export, which keeps structural tags such as `<heading>`, `<fnote>`, `<list>`, `<figure>` and `<quote>`. About 88 percent of the human essays in my sample contain at least one of these tags. None of the AI essays do, because Llama writes prose, not corpus markup. A rule as simple as "if the text contains a tag, call it human" reaches 92.5 percent accuracy on the test set on its own, with no understanding of language at all. The AI side has the mirror version of the same problem: Llama sometimes writes markdown (bold, headings, bulleted and numbered lists) that human plain text never has. Both of these are artefacts of how the two halves of the corpus were produced, not differences in writing, and they sit right at the start of the text where the model's 512-token window sees them. This is the shortcut behind a large part of the perfect score.

**Does anything real remain once the artefact is removed?** Yes. I wrote a normaliser (`src/detection/text_normalize.py`) that strips the BAWE tags from the human side and the markdown from the AI side and flattens the layout, then rebuilt the corpus from the cleaned text. On the cleaned text a simple TF-IDF and logistic-regression model still separates the two classes at 100 percent on the test set. I treat that 100 percent with some caution, because one residual markup token ("`fnote`") survived the cleaning into the full model's

vocabulary. I traced it: the cleaning strips the bracketed tags, but 115 of the 640 AI essays contain the bare word "fnote", echoed by the generator from keyword prompts that were extracted from the human source text. So in the cleaned corpus "fnote" flips from a human marker to a weak AI one; it is a generation artefact rather than leftover markup, it does not touch the function-word or stylometric results, and stripping the bare token from both sides is queued for the next retraining pass. The cleanest evidence is therefore the next result, not this one. A model that is allowed to use only function words (the, and, because, therefore, and the like, which carry no topic information at all, and no markup) still reaches 99.5 percent. Topic cannot explain that, and neither can a leftover tag. The thing that remains after the markup is gone is writing style.

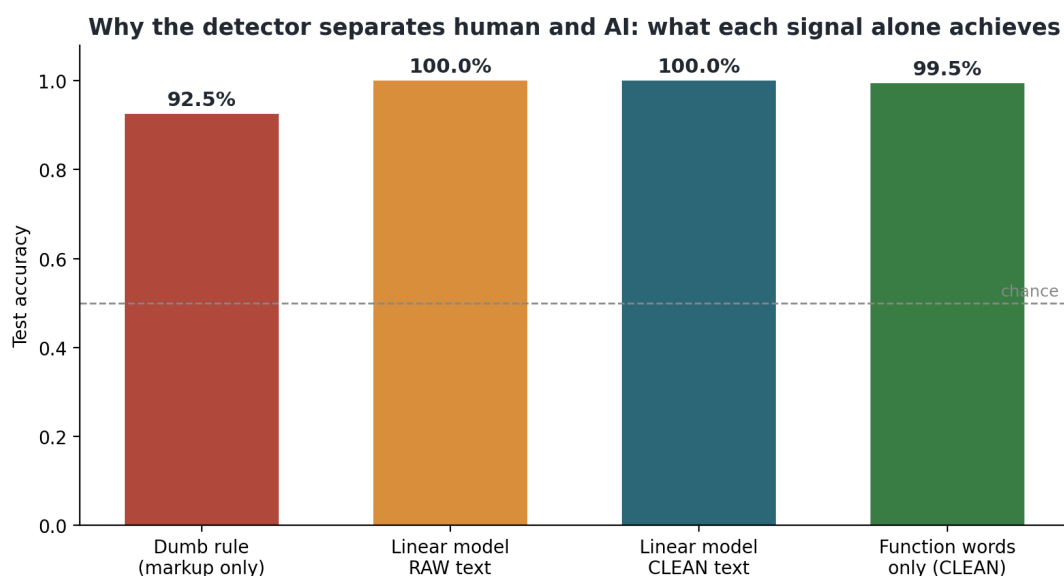


Figure 3.1: What each signal achieves on its own. A markup-only rule already scores 92.5 percent on the raw text, which is the size of the artefact. After cleaning, a simple model still separates the classes, and a function-words-only model still reaches 99.5 percent, so the residual signal is writing style.

**What is the model keying on?** The cleaned linear model is interpretable, so I can read off the words that push each way. The AI-leaning terms are the familiar large-model register: "in conclusion", "nuanced", "essential", "highlights", "insights", "complex", "significant". The human-leaning terms are blunter connectives: "therefore", "because", "so", "thus", "very". That matches what the stylometric features already showed, that human writing here varies more and the AI writing is smoother and more uniform.

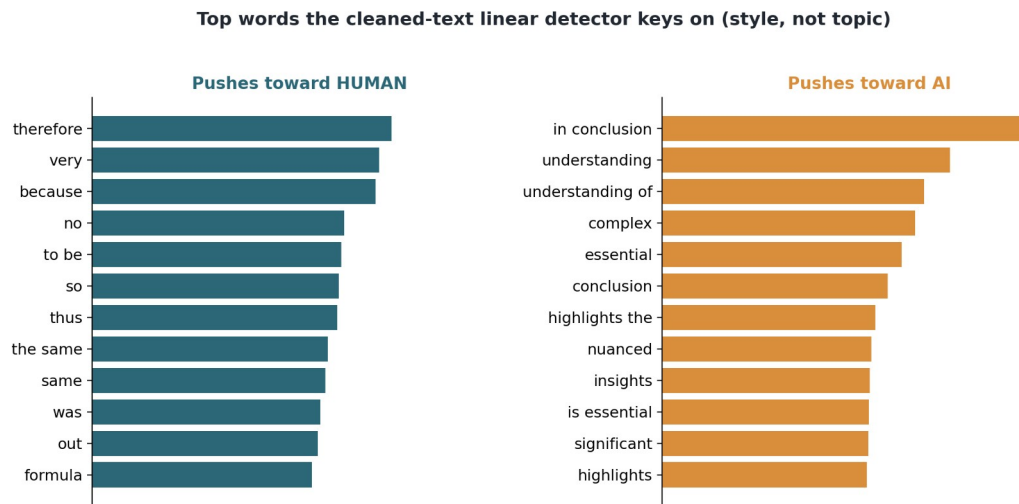
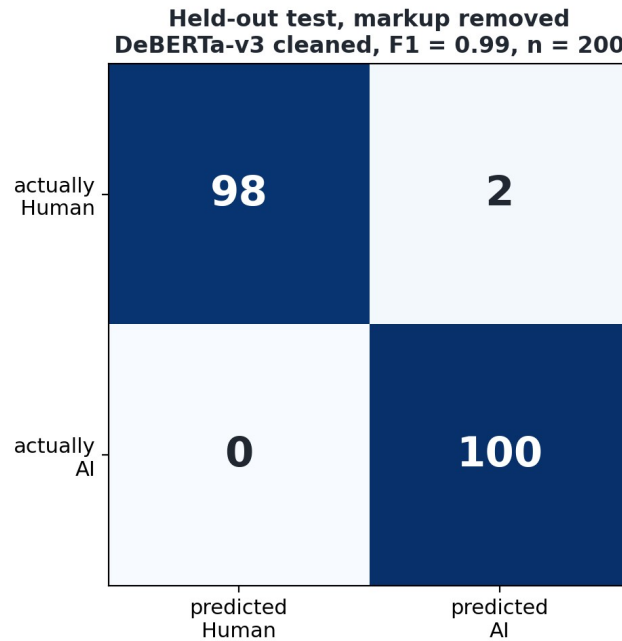


Figure 3.2: The words the cleaned-text model keys on. The AI side is the familiar large-model register; the human side is blunter argument connectives. These are the seeds of the explainability layer.

### 3.6 The honest headline

After removing the artefact I retrained DeBERTa on the cleaned corpus. The score moved off the ceiling, which is what I wanted: test accuracy 0.99, precision 0.98, recall 1.00, F1 0.990, with a confusion matrix of  $\begin{bmatrix} 98 & 2 \\ 0 & 100 \end{bmatrix}$ . Two human essays are flagged as AI and no AI essay is missed. The test set is small (100 human and 100 AI essays), so I put a number on that uncertainty: a 95% bootstrap confidence interval on the F1 is about  $[0.97, 1.00]$  (`src/evaluation/confidence.py`, Figure 3.5). One extra misclassification moves the F1 by roughly 0.005, so the headline should be read as "about 0.99" rather than an exact value. The full-document linear probe still scores 1.00 on the same cleaned text, while DeBERTa makes two mistakes, which fits the fact that the transformer only reads the opening 512 tokens while the linear model sees the whole essay.

On fairness, the human false-positive rate is 2 of 50 native writers (0.04) and 0 of 50 non-native writers (0.00). With only 50 humans per group, one essay is worth 0.02, so this is not enough to establish a fairness gap, and I treat it as indicative only. The stronger fairness evidence is the cross-domain result in Chapter 6, where the human false-positive rate is measured on far more text and shows the real bias mechanism.



*Figure 3.3: DeBERTa on the held-out test set after the markup was removed. Two human essays are flagged as AI and no AI essay is missed (F1 0.990).*

RoBERTa, trained the same way on the cleaned corpus, lands in the same place: F1 0.995, a confusion matrix of  $[[99, 1], [0, 100]]$ , native false-positive rate 0.02. Its only difference from DeBERTa is one human essay flipping, and the two confidence intervals overlap almost completely (Figure 3.5), so on this 200-essay test set 0.990 and 0.995 are statistically indistinguishable and I do not read anything into the gap. What is meaningful is that both architectures land in the same regime: high accuracy, and every mistake is a human essay flagged as AI rather than an AI essay missed. That shared one-sided error pattern, not the near-identical scores, is the sign that the residual signal is a real style difference and not a quirk of one model.



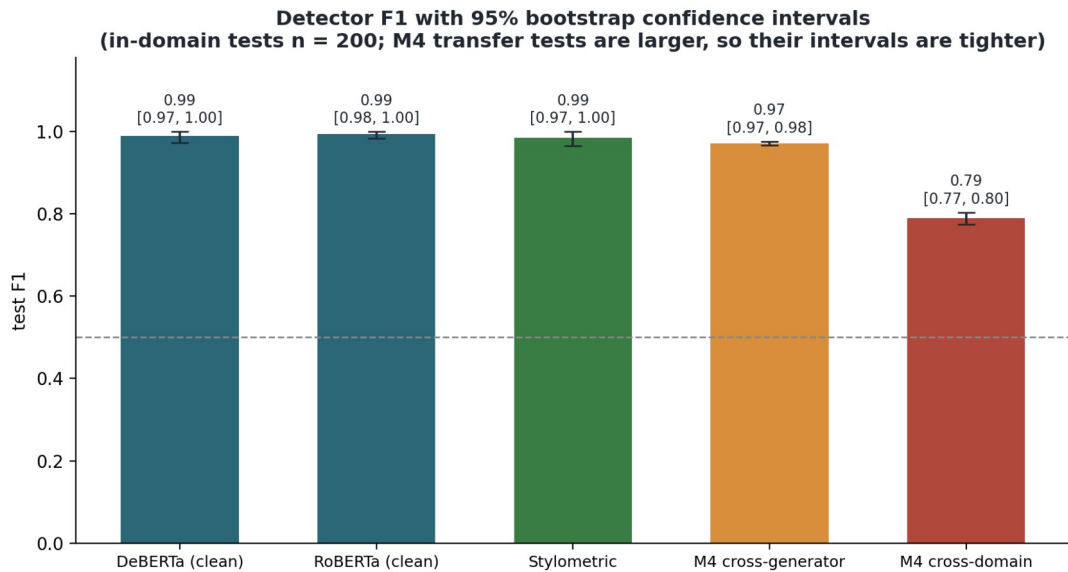


Figure 3.5: Test F1 with 95% bootstrap confidence intervals. The in-domain detectors (DeBERTa, RoBERTa, stylometric) have heavily overlapping intervals on  $n=200$ , so their differences are within sampling noise; the M4 transfer results have tight intervals on much larger samples.

So the cleaned F1 of 0.990 is the number I report, and the raw 1.000 is kept only to show how large the artefact was. Either way the in-domain task is easy, which is consistent with the literature: separating one known generator from human writing in one domain is close to a solved problem. That is not the contribution of this project. The contribution is the explanation behind each decision and, after this stage, robustness to the harder settings that matter in practice, which are other generators, paraphrased AI text, and submissions that mix human and AI writing. I expect the score to drop there, and that drop is where the real research is.

### 3.7 Why the score is still high after cleaning

A fair question is why a cleaned score of 0.99 is not itself suspicious. I dug into this (`src/detection/why_high.py`) so I can answer it rather than wave it away. The score is high because the task as I have built it is the easy version of the problem, and several separate signals reinforce each other.

The biggest reason is that all of my AI text comes from one model with one prompt. The AI class is really "Llama 3.1 writing the way I asked it to", so it has a consistent style. When I measure how similar essays are to each other in style space, the AI essays are more alike (average similarity 0.60) than the human essays (0.56), because the humans are a crowd of different people and the AI is one voice repeated 640 times. Separating one consistent voice

from a varied crowd is not hard, and the detection literature treats single-generator, in-domain detection as close to solved.

On top of that sit several smaller tells that all point the same way. The vocabulary differs (the AI register I described above). The spelling differs: the human essays in my sample are in British English (about 2.3 British spellings per thousand words) and Llama defaults to American (about 2.4 American spellings per thousand words). This one is real but shallow, and I am honest that it is a generator-locale artefact rather than deep evidence of AI authorship; it would shrink if I prompted the model to write British English, which is a fix I will try. Formality differs too: the human essays use about three times as many contractions. None of these on its own is decisive, but they stack, and a 2D picture of the essays in pure style space (Figure 3.4, function words only) shows two clouds that barely touch. So 0.99 is the expected result for an easy setting, not a hidden error.

I checked how much of this is the shallow locale tell rather than deep style (`src/detection/style_locale_control.py`). When I remove every British and American spelling variant and every contraction from the text, which changes 1,251 of the 1,280 essays, the function-words-only accuracy does not fall: it goes from 0.995 to 1.00, and the full TF-IDF model stays at 1.00. So the locale and formality tells are real but they are not what the separation rests on; the distributional style signal is there with or without them. I keep the locale point as an honest caveat and a planned fix, not as the explanation for the score.

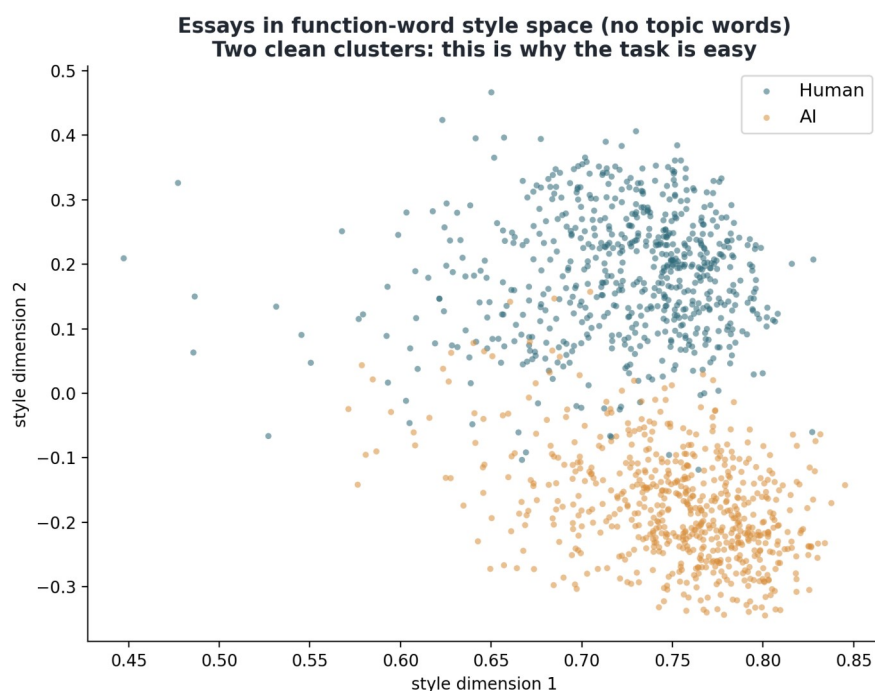


Figure 3.4: Every essay placed by its function-word style alone, with all topic words removed. Human and AI fall into two separate clouds, which is why a single generator in one domain is easy to separate.

The closest call is reassuring rather than worrying. The human essay the model rates most AI-like scores 0.43 on the AI scale, so it is still correctly called human, and the borderline cases are a small mix of native and non-native writers rather than a pile of non-native essays, which is the bias I was watching for.

### 3.8 What this means for the next steps

The audit also seeds the explainability layer. The system can already point at the words that drove a decision, which is the first concrete version of the defensible evidence the whole project is built around. The immediate next steps are to fuse the stylometric features with the transformer, to test the detector against the M4 multi-generator benchmark so I have an honest cross-generator number, and to build the attribution methods (Integrated Gradients and SHAP) on top of the model so the word-level explanation is faithful and not just a linear proxy.

## Chapter 4: Implementation

### 4.1 Environment and reproducibility

The whole project runs on my own laptop. After Meeting 3 it became clear that no ATU HPC or cloud would be available, so every part has to fit a single RTX 4060 with 8 GB of VRAM, or fall back to the CPU. That constraint shaped a lot of the choices below, and it is the reason the models are base-sized rather than large.

The stack is Python on Windows, with PyTorch and the Hugging Face libraries for the transformer work, spaCy for the linguistic features, and scikit-learn for the simpler baselines and the audit probes. To keep runs repeatable I set a single random seed (42) everywhere that randomness enters: the sampling, the train and test split, and the model training. Dependency versions are pinned, and the work is committed to a local git repository in small steps with clear messages, so any result can be traced back to the exact code that produced it. Long jobs (essay generation, model training) are launched as detached background processes rather than tied to an open terminal session, so they keep running if the session that started them closes. This sounds like a small detail, but it cost me two failed generation runs before I worked it out, and it is now the standard way I start anything that takes more than a few minutes.

### 4.2 Data acquisition and cleaning

The human side of the corpus is the British Academic Written English (BAWE) collection, which I downloaded from the Oxford Text Archive. A first script (`src/data/explore_bawe.py`) reads the holdings spreadsheet, prints a summary, and cross-checks the recorded word counts against a plain count of the text files so I know the metadata is trustworthy. A second script (`src/data/clean_bawe.py`) produces a cleaned metadata table, dropping a row that was labelled twice. The cleaned table is what every later step samples from.

### 4.3 Sampling

The sample is drawn by `src/data/build_sample.py`. Rather than sample by named discipline, I stratify evenly across the four broad disciplinary groups and balance native against non-native English writers, so that I can later test whether the detector is unfair to non-native writers. A per-student cap stops any one person's style from dominating, and the train,

validation and test split is made at the level of the student, not the essay, so the same writer never appears on both sides of the split. The result is 640 human essays with a versioned manifest (`bawe_human_sample_manifest.csv`) that records, for each essay, its group, first-language status, and split. The reasoning behind the sizes is written up in `dissertation/sample_design.md`.

#### 4.4 Generating the matched AI essays

The AI half of the corpus is built by `src/generation/generate_ai_essays.py`. For each human essay it generates one AI essay on the same topic and at the same target length, using Llama 3.1 8B running locally through Ollama. The length and topic matching is the most important part: if the AI essays were systematically longer or shorter, or drifted onto different subjects, the detector could separate the classes for the wrong reason and the whole corpus would be invalid.

Two things make the matching work. First, the prompt is anchored on keywords pulled from the human source text, not just the essay title, after an early test produced an off-topic essay from a title alone. Second, the generator runs a short continuation loop: if the first draft falls short of the target length it asks the model to continue, up to a few rounds, so the AI essay lands close to its human counterpart. The script is resumable, so a run that stops can be restarted and it skips what is already done, and it keeps the machine awake while it runs. It writes each essay to disk along with a metadata row recording the target and actual length, the number of rounds, and the time taken.

Generation of all 640 essays completed locally with no failures. A validation script (`src/generation/check_ai_corpus.py`) confirmed the match: the correlation between each AI essay and its human source length is about 0.98, the mean length ratio is about 1.05, and over 99 percent of AI essays are within 20 percent of their source length. A keyword spot-check confirmed the AI essays stayed on the same topics. So the two halves differ in writing, not in length or subject, which is exactly the property the detector needs.

#### 4.5 Building the labelled corpus

`src/detection/build_detection_corpus.py` pairs each human essay (label 0) with its matched AI essay (label 1), carries the split and metadata across from the manifest, and writes a single table of 1,280 rows. The same script has a cleaning mode (described in Section

4.7) that strips markup before writing, so the raw and cleaned corpora can be compared directly.

## 4.6 The detector

The detector is fine-tuned by `src/detection/train_detector.py`. It uses the Hugging Face Trainer to fine-tune a transformer to classify human against AI, with DeBERTa-v3-base as the primary model and RoBERTa-base as a comparison. To fit the 8 GB card I keep the per-step batch small and use gradient accumulation to reach a sensible effective batch, with mixed precision on the GPU. The script reports accuracy, precision, recall, F1 and the confusion matrix, and on top of that it breaks out the false-positive rate for native and non-native writers separately, which is the seed of the fairness analysis. The stylometric feature extractor (`src/detection/stylometric.py`) computes the linguistic features (sentence-length variation, vocabulary richness, part-of-speech mix, and so on); fusing those with the transformer is the next step and is not part of this first detector.

## 4.7 The audit and the cleaning step

The first detector scored a perfect 100 percent, which I did not trust. The audit that followed is implemented in three small scripts, and they are part of the contribution, not throwaway checks.

`src/detection/audit_detector.py` runs the diagnostic battery: it confirms there is no student or duplicate-text leakage across the splits, measures how much of the score a markup-only rule can reach on its own, and trains simple interpretable baselines on raw and cleaned text, including a function-words-only model that removes all topic information. `src/detection/text_normalize.py` is the cleaning function that the audit relies on: it strips the BAWE export tags from the human text and the markdown from the AI text, and flattens the layout, so that any remaining difference is writing style rather than formatting. `src/detection/why_high.py` then explains why the cleaned score stays high, by measuring locale spelling, contraction use, and how tightly the AI essays cluster in style space, and by drawing the two classes in a two-dimensional style projection.

The finding, covered in Chapter 3, was that the original human text carried structural tags that no AI essay had, and that this shortcut accounted for a large part of the perfect score. The

cleaning step removes it, and the detector is then retrained on the cleaned corpus to give the honest headline figure.

#### **4.8 Engineering notes and lessons**

Two practical lessons are worth recording for the methodology. The first is the value of launching long jobs as detached processes, which is what finally made the overnight generation reliable. The second is the markup artefact itself: it is a clean example of why a strong result has to be stress-tested before it is believed, and the fact that the audit is scripted means anyone can rerun it and see the same thing. Both points feed directly into how I will build and check the remaining components.

## Chapter 5: Explaining the detector's decisions

### 5.1 Why this matters for the project

The whole point of the project is that a detector should hand a lecturer defensible evidence, not just a percentage. So the detector is only half the story. The other half is being able to say why a piece of text was flagged, and being honest about whether that explanation is real. This chapter is the first version of the explainability layer. It does two things: it shows which parts of an essay drove the decision, and then it tests whether that explanation is faithful, which means whether the words it points to are actually the words the model used.

### 5.2 Token attributions with Integrated Gradients

For the word-level view I used Integrated Gradients, a standard attribution method, through the Captum library (`src/explainability/integrated_gradients.py`). It works on the detector's word embeddings and measures how much each token pushes the decision toward the AI class, compared to a neutral baseline where the content is removed. I attribute everything to the AI class so the sign is consistent: a positive score pushes toward AI, a negative score pushes toward human.

Figure 5.1 shows this for a matched pair, the AI and human versions of the same essay. Two things stand out. The detector is confident and in the right direction for both (the AI essay scores 1.00 on the AI scale, the human essay 0.01). And the tokens that carry the most weight are not topic words. They are punctuation and very common words and word-fragments: full stops, commas, a dash, words like "have", "be", "can", "similarly". That fits everything from the audit. The model is reading style and rhythm, the small machinery of how a sentence is put together, rather than what the essay is about.



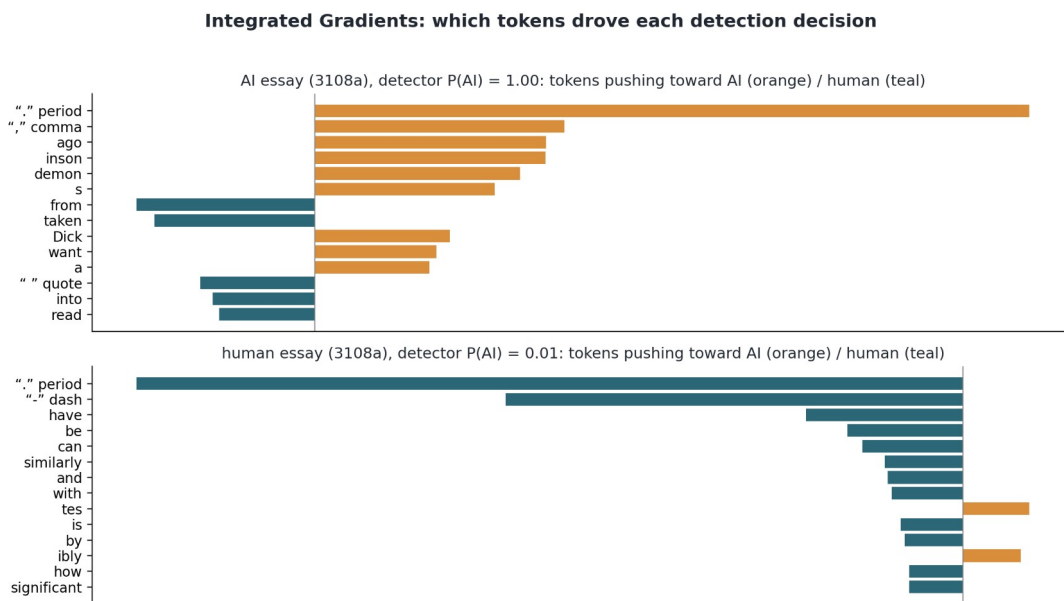


Figure 5.1: Integrated Gradients token attributions for a matched AI and human essay. Orange pushes toward AI, teal toward human. The strongest tokens are punctuation and common words, not topic words, so even at the word level the detector is reading style.

### 5.3 Are the highlights honest? A faithfulness test

A highlight is only useful if it is faithful. I tested this by ablation (the same script). For a sample of test essays I ranked the tokens by their attribution and then asked two questions. First, comprehensiveness: if I remove the top tokens, does the detector's confidence fall, and does it fall more than if I remove the same number of random tokens? Second, sufficiency: if I keep only the top tokens and hide the rest, does the confidence hold up?

Figure 5.2 shows the comprehensiveness result as I remove more and more tokens. Removing the top-ranked tokens does hurt the detector more than removing random ones, but only by a little, and only once a fair number are removed. This comprehensiveness sweep is the load-bearing, honest part of the test. I also ran a sufficiency check (keep only the top tokens, hide the rest), but with at most a few dozen of 256 tokens kept the model sees almost nothing and sits at a coin-flip for every  $k$ , so that flat result is an empty-input artefact of the design rather than a finding, and I do not rely on it. The comprehensiveness result on its own already says what matters: no small set of words is enough to carry the decision.

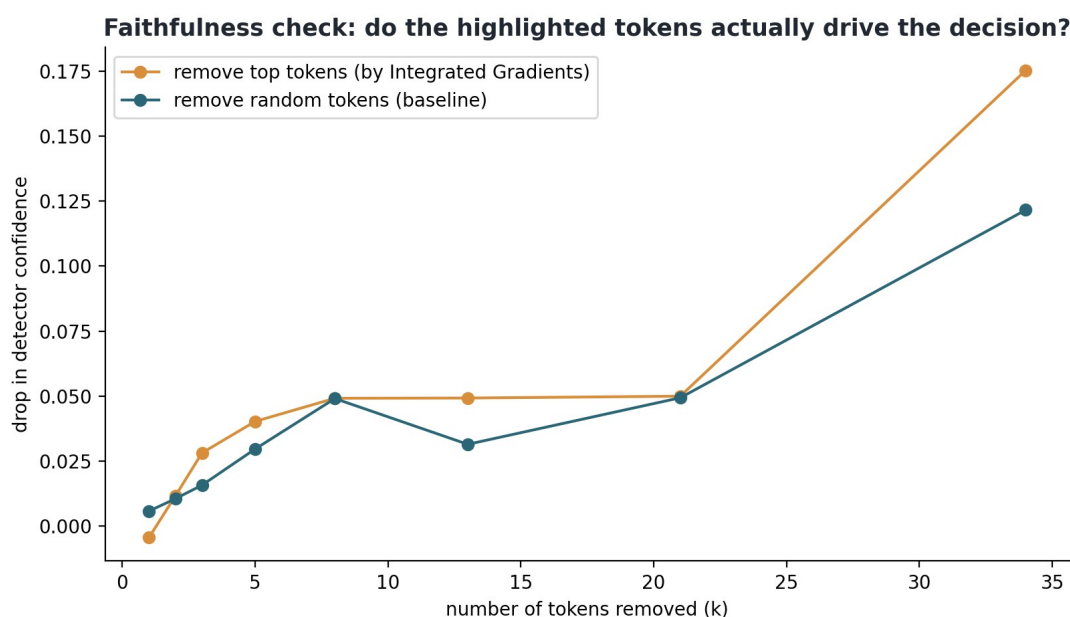


Figure 5.2: Faithfulness by ablation. Removing the top attributed tokens (orange) lowers detector confidence only slightly more than removing random tokens (teal). Keeping only the top tokens collapses the prediction to chance. The signal is spread across the whole essay.

## 5.4 What this tells me: the signal is diffuse

This is an honest and useful negative result. The reason a few words cannot explain the decision is that the difference between human and AI writing here is spread across the whole essay, in the ordinary words and the punctuation, not concentrated in a handful of give-away terms. It is the same thing the audit found from the other direction: a model using only function words still separated the classes at 99.5 percent, and the essays form two clean clouds in pure style space. A diffuse signal is hard to summarise with a word-level highlight, so for this detector a per-word heatmap is a weak explanation on its own.

The explanation that is faithful for this detector is the feature-level one. The interpretable linear view from the audit (Chapter 3) names the style markers that separate the classes, the large-model register on one side and the blunter connectives on the other, and that view is backed by the function-words-only result rather than contradicted by it. So the defensible, lecturer-facing explanation is the style and feature picture, supported by the source-grounded verification questions that come later in the pipeline, not a single highlighted sentence.

## 5.5 The faithful explanation: stylometric features and SHAP

To make the feature-level explanation concrete I built a detector from hand-crafted style features alone, with no transformer: sentence-length variation and burstiness, vocabulary

richness, word length, punctuation, and the part-of-speech mix (src/explainability/shap\_stylometric.py). Trained on the same student-level splits, this transparent model reaches an F1 of 0.985 on the test set (95% bootstrap confidence interval about [0.97, 1.00], and identical, 0.985, across five training seeds, so it is stable). Two honest caveats. The comparison with the transformer's 0.99 is not quite like for like: the feature model reads the whole essay while DeBERTa reads only the first 512 tokens, which helps the feature model, so I read this as "competitive" rather than "equal". And the false-positive rate of 0.02 for native and non-native writers is one essay in fifty each, too small to claim fairness; the cross-domain result in Chapter 6 is the real fairness evidence. Two of the features (type-token ratio and the rare-word ratio) are length-sensitive, but since the essay lengths are matched across the classes this carries little class signal; the length-robust vocabulary measure behaves the same way. Perplexity, one of the strongest stylometric signals in the literature, is implemented but not yet in this model (it needs a GPU pass) and is deferred to the hybrid. Even so, the headline holds: the signal is captured almost as well by a handful of interpretable features as by a large model.

Because the model is built from named features, I can explain it faithfully with SHAP, which attributes each decision to the features that drove it. Figure 5.3 shows the picture across the test set. Longer words and a denser use of auxiliary verbs push a text toward AI, while more sentence-length variation, richer vocabulary, and more rare one-off words push it toward human. This is the explanation the project actually needs: a lecturer can be told that a piece was flagged because its sentences are unusually uniform, its words longer, and its vocabulary less varied than a typical student's, and that account is faithful because the model literally uses those features. It is the opposite of the black-box percentage the project set out to replace.

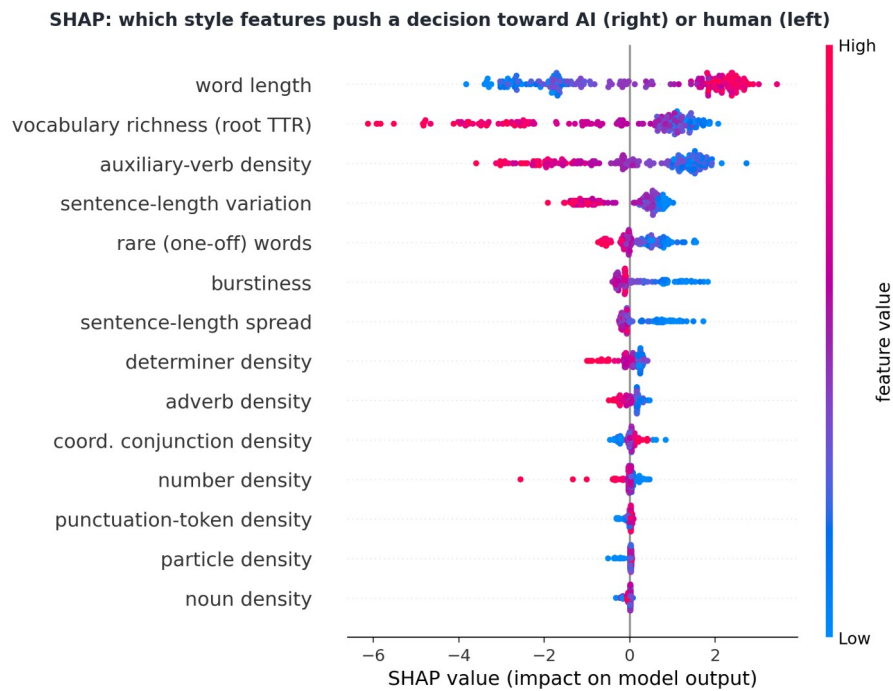


Figure 5.3: SHAP on the stylometric detector. Each dot is an essay; position shows how much a feature pushed the decision toward AI (right) or human (left), and colour shows the feature value. Longer words push toward AI; richer vocabulary and more varied sentence length push toward human.

This stylometric model is also the non-transformer half of the planned hybrid detector, so this step serves both the explainability layer and the detector itself.

## 5.6 Next steps for explainability

Three things follow. First, fuse the stylometric model with the transformer into the hybrid detector, and add perplexity (a strong feature that needs the GPU) to the feature set. Second, keep the token attributions as a supporting visual, useful for showing a lecturer roughly where the model looked, but always paired with the honest caveat from the faithfulness test, with the SHAP feature view as the primary explanation. Third, carry the faithfulness check forward as a standard step: any explanation the system shows should be one I have tested, not one I have assumed.

## Chapter 6: Robustness, transfer to unseen generators and domains

### 6.1 The question

The in-domain detector scores about 0.99, but that is on the easy setting: one generator (Llama) and one kind of text (student essays). The honest question is what happens when neither holds. To test it I took the trained detector, with no retraining or adaptation, and ran it on the M4 benchmark (SemEval-2024 Task 8), which has many generators and many domains that the detector never saw. I split this into two tests so I do not accidentally call the easy half "robust".

A note on sources, so the two tests are not confused. Test A uses the OUTFOX essay set that ships inside the M4 release: human essays against six generators. Test B uses the M4 / SemEval-2024 Task 8 monolingual data across five web and academic domains. They are different corpora with different generator sets, which I chose deliberately: A isolates a change of generator while holding the text type fixed (essays), and B changes the domain. I name them separately rather than calling both simply "M4".

### 6.2 Test A: transfer to unseen generators (still essays)

The first test uses the OUTFOX essay set, where humans and six different models (GPT-4, ChatGPT, Cohere, BLOOMz, Dolly, and davinci) all wrote essays. This keeps the kind of text the same as training and only changes the generator. The detector held up well: an F1 of 0.97 (95% confidence interval [0.97, 0.98], on about 4,800 essays so the interval is tight), and it caught every generator at between 96 and 100 percent (Figure 6.1), with a human false-positive rate of about 5 percent. This is a useful and slightly surprising finding. A detector trained only on Llama still recognises text from quite different models. The reason is that current large models share a lot of the same smooth, uniform style, so the AI fingerprint is largely generator-agnostic, at least on essays.

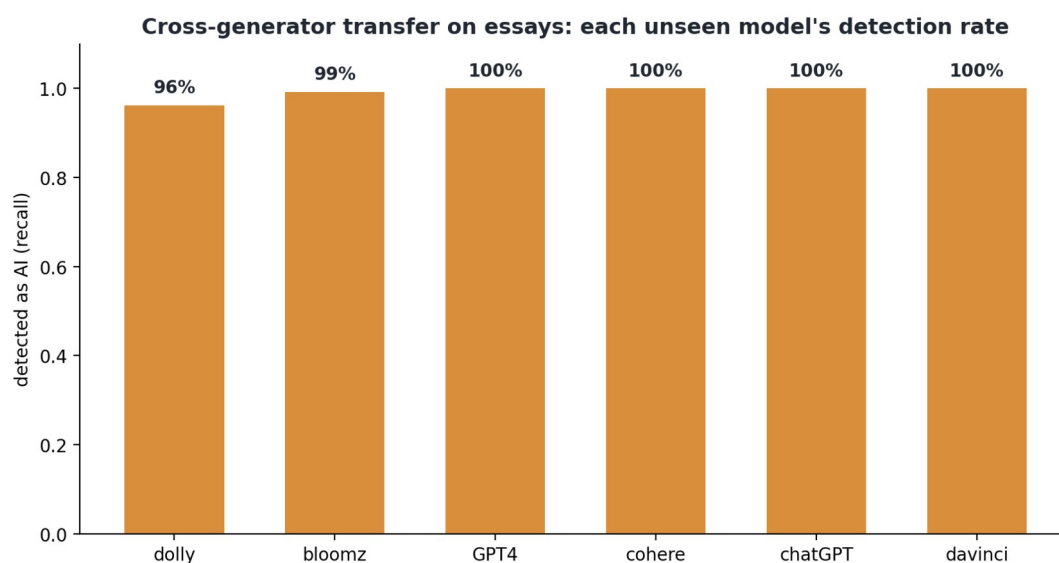


Figure 6.1: Transfer to unseen generators on essays. A detector trained only on Llama still flags GPT-4, ChatGPT, Cohere, BLOOMz, Dolly and davinci at 96 to 100 percent.

### 6.3 Test B: transfer to unseen domains

The second test is the hard one, on the M4 monolingual data. Here the human text is not essays at all: it is Reddit posts, WikiHow articles, arXiv abstracts, Wikipedia, and peer-review text, against four generators. The overall F1 falls to 0.79 (95% confidence interval [0.77, 0.80], tight because the sample is large), and the way it fails is the important part. The generator set here differs from Test A, so the headline drop mixes a change of domain with a change of generators. The load-bearing evidence is therefore not the combined F1 but the human side of the result, because a false-positive rate measured on human text cannot be a generator artefact. The detector still catches the machine text well in every domain (86 to 98 percent). What breaks is the human side: it wrongly flags genuine human writing as AI at high rates on the more formal domains, 79 percent on arXiv abstracts, about 40 percent on Wikipedia and WikiHow, 30 percent on peer-review, and 23 percent on Reddit (Figure 6.3). That human-side failure is what carries the argument.

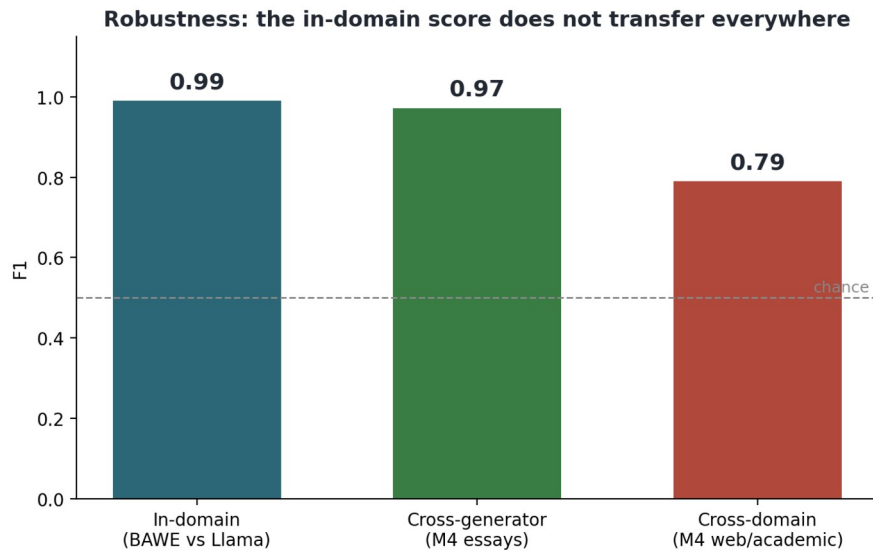


Figure 6.2: The in-domain score does not transfer everywhere. F1 stays high across generators on essays but falls on out-of-domain human and AI text.

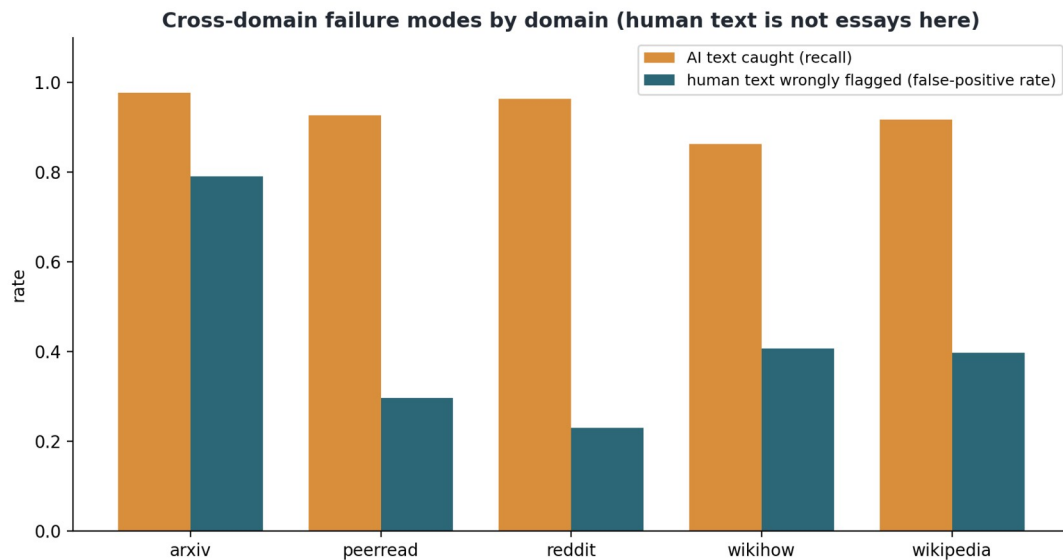


Figure 6.3: Cross-domain failure modes. The detector still catches AI text (orange) but wrongly flags formal human text as AI (teal), worst on arXiv abstracts.

## 6.4 What this means

The picture is clear and it matters for the project. The detector is robust across AI models but fragile across domains, and the fragility is not random. It learned that "human" looks like a student essay, so when it meets human writing that is more formal or more technical, an arXiv abstract for example, it calls it AI. In other words, the out-of-domain failure is a false-accusation failure, exactly the harm the project is trying to avoid.

This connects directly to the fairness concern from the start of the project. A detector trained on one narrow idea of human writing will misjudge writers whose style sits outside that idea.

The arXiv result is the clearest warning: formal, dense, careful human prose reads as machine to this model. The same mechanism is what makes these tools risky for non-native writers, whose style also differs from the training norm. It is also the strongest argument for why the pipeline does not stop at a score. A number that is confidently wrong on a whole domain is dangerous on its own; the verification questions are what turn a flag into something a lecturer can check rather than act on blindly.

## **6.5 Caveats and next steps**

This is a zero-shot test with no adaptation, so it is a lower bound on what is achievable: a detector trained on diverse human text and several generators would almost certainly do better across domains, and that is a clear next experiment (train on a slice of M4 and re-test). The in-domain "human" being essays is the root of the false positives, so the fix is more varied human data and per-domain calibration. The remaining robustness gaps to test are paraphrased and "humanised" AI text, and documents that mix human and AI writing. For the project as it stands, the honest headline is that the detector is good at spotting AI essays from many models, and not safe to point at human writing from a domain it was not trained on.



## Chapter 7: From a flag to verification questions (first slice)

### 7.1 Why this is the point of the project

Detection on its own only produces a flag, and the earlier chapters showed how fragile a flag can be out of domain. The actual contribution of the project is what happens after the flag: turning a suspected submission into something a lecturer can act on fairly, which is a short set of questions, drawn from the student's own claims, that check whether the student understands what they handed in. A student who wrote and understood the work can answer them; a student who did not will struggle. This chapter is the first working version of that step.

### 7.2 How the slice works

The pipeline takes a flagged essay and produces a Verification Interview Guide (`src/question_gen/generate_questions.py`). It runs in four steps.

First, it splits the essay into numbered sentences. Second, it extracts the student's main claims, and for each claim it records the sentence numbers the claim comes from. This is the important design choice for provenance: the model is asked to cite sentence numbers, not to quote the text, so the source is looked up from the real numbered sentences and the model cannot invent a quote. Every claim therefore points at exact lines in the submission. Third, for each claim it generates verification questions that are grounded in the source sentences and written so that they cannot be answered well by someone who only read the claim. Fourth, each question is tagged with a Bloom's cognitive level.

Backends sit behind one interface. This slice runs on the local open-source model (Llama 3.1 8B through Ollama), which is the basis for Backend B. The commercial Backend A plugs into the same interface once an API key is available, and because the backend is recorded with every guide the two can be compared directly, which is the core commercial-versus-local research question.

### 7.3 An example

Run on a flagged essay that compares three literary texts, the system pulled out four claims and wrote grounded questions for each. Two of them give the flavour.

For the claim that the three texts use language differently (metaphor and symbolism in the poetry, dramatic intensity in the drama, subtle nuance in the prose), tied to two specific

sentences in the submission, it asked: "What specific features of Shakespeare's dialogue lead you to describe it as having dramatic intensity?" and "How does Austen's subtle and nuanced prose style allow her to explore complex social issues in a way that might be less effective with more overt language?" Both need the student to reason about the text, not repeat the claim.

For the claim that the texts share themes of love and personal growth, tied to a single sentence, it asked for an example from one of the texts that links personal growth to relationships, which is a question a student who genuinely engaged with the texts can answer and a student who did not cannot. The full guide, with every claim, its exact source sentences, and the Bloom's levels, is saved as `outputs/verification_guides/3108a_ai.md`.

## **7.4 What is first-slice and what comes next**

This is deliberately a thin path through the remaining pipeline, and I am honest about the parts that are stand-ins. The claim extraction is currently prompted rather than the trained argument miner planned in the scope (the Persuasive Essays corpus, claim and premise labelling); a prompted extractor can miss or merge claims, but it feeds question generation now and can be upgraded later. The Bloom's tag is a transparent keyword heuristic, a placeholder for the BERT classifier that is its own component. And the questions have not yet been evaluated: the planned discrimination simulation, where a model with the source and a model without it both try to answer each question and the gap shows how well the question targets understanding, is the next piece of work and is what will turn these example questions into measured results.

The immediate next steps are therefore to add the commercial backend and run the commercial-versus-local comparison, to stand up the discrimination-simulation evaluation, and to decide with my supervisor whether the local backend is the full Llama 3 8B with QLoRA or a smaller model that fits the laptop, since that choice affects the comparison.

## **7.5 The Bloom's classifier: replacing the heuristic**

The keyword heuristic above was always a placeholder, and component 5 of the scope is a trained classifier. I fine-tuned BERT-base on the Bloom-labelled subset of EduQG (Hadifar et al., 2022): 903 questions carrying a cognitive level, of which only four levels occur (remember, understand, apply, analyse) in a heavily skewed distribution (660, 114, 110 and 19 examples

respectively). Training used class weights against that imbalance, stratified splits (632 train, 135 validation, 136 test), and six epochs; the transparent keyword heuristic was scored on the same held-out test split as the baseline.

The trained model roughly doubles the heuristic: macro-F1 0.31 against 0.16, accuracy 0.57 against 0.26 (Figure 7.1). The per-class picture is the honest part. BERT is reliable on the majority class (remember, F1 0.76) and moderate on understand (0.38), but it cannot learn apply (0.09) or analyse (0.0) from 110 and 19 examples. Coarsening the same predictions into the standard lower-order versus higher-order split does not rescue the minority side either: higher-order F1 is 0.23 for BERT and 0.17 for the heuristic, and the heuristic's apparently high binary accuracy (0.86) is majority-class bias rather than skill, since predicting lower-order for everything scores about 0.85 on this test set.

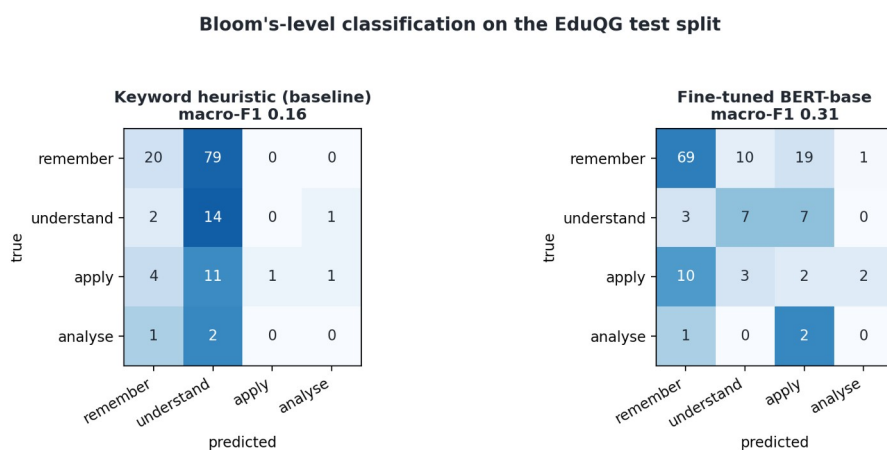


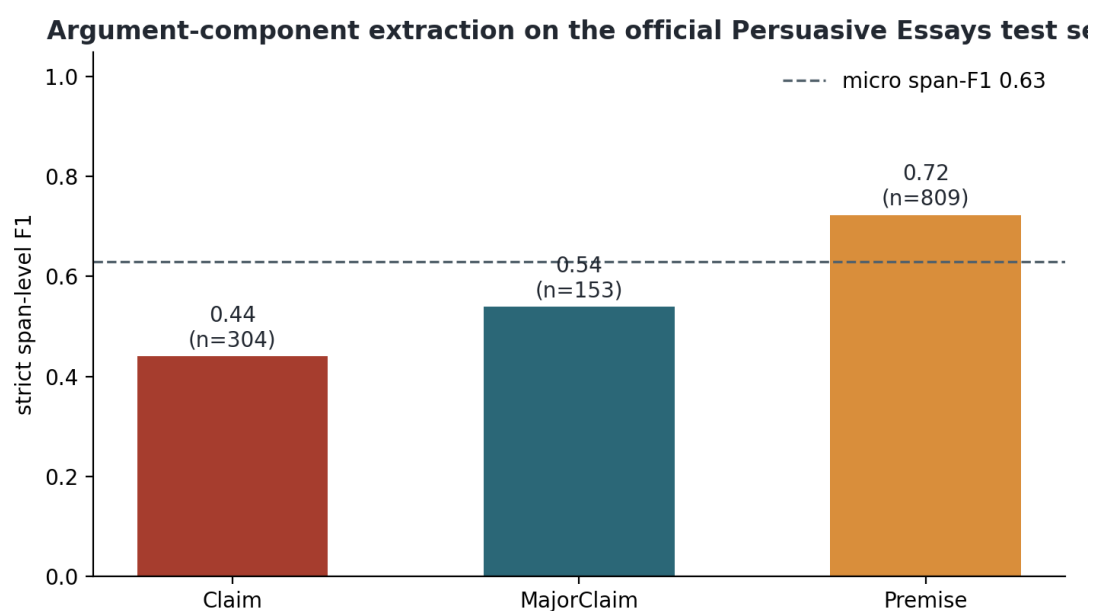
Figure 7.1: Bloom's-level classification on the EduQG test split. The trained BERT-base doubles the keyword heuristic on macro-F1 (0.31 vs 0.16), but neither model can learn the two smallest classes from 110 and 19 examples.

Two conclusions follow. The component works and clearly beats the transparent baseline, so it replaces the heuristic in the pipeline. And the labelled data, not the model, is the bottleneck: 73 percent of EduQG's labels are the lowest level, so a classifier that must recognise higher-order questions needs either more labelled examples of them or a coarser, better-populated label scheme. For the guide's quality-control purpose the practical reading is that Bloom's tags on remember and understand questions can be trusted, and tags on the higher levels should be treated as suggestions until the label supply improves. Both the model and the full metrics are saved (models/bloom\_classifier/, outputs/bloom\_classifier.json).

## 7.6 The trained claim extractor

The prompted claim extraction above was the declared stand-in for component 3, and the trained version now exists. I downloaded the Persuasive Essays 2.0 corpus from its official repository (402 essays, BRAT annotations for major claims, claims, and premises, with the corpus's own 322/80 train/test split) and fine-tuned DeBERTa-v3-base as a BIOES-style token classifier over the three component types. Sequences are paragraphs, since no annotated component crosses a paragraph and the longest paragraph fits inside 256 subwords, so nothing is truncated; evaluation is strict span-level (exact boundary and exact type) with sequence-level on the held-out test essays.

The result is a micro span-F1 of 0.63: premises 0.72, major claims 0.54, claims 0.44 (Figure 7.2). Strict matching is a hard yardstick, and the per-class order is the expected one, since claim boundaries are the classic ambiguity in this corpus. Dedicated argument-mining architectures with CRF decoding or joint relation modelling report higher figures, so this is a working first version rather than the state of the art, and I report it as such. For the pipeline it opens a design choice to settle with my supervisor: the trained extractor finds spans in the student's own words, while the prompted extractor produces readable claim paraphrases with sentence citations, so the natural combination is spans for provenance and the prompt for phrasing. The model and metrics are saved (models/claim\_extractor/, outputs/claim\_extractor.json).



*Figure 7.2: Argument-component extraction on the official Persuasive Essays test set, strict span-level F1 per class. Premises are learned well; claim boundaries are the hard case, matching the corpus literature.*

## 7.7 The assembled guide: the pipeline's output document

With the classifier in place, the output assembler (`src/pipeline/assemble_guide.py`) now produces the document the pipeline exists for: a Verification Interview Guide a lecturer can take into a conversation. For a submission it runs the trained detector live and reports the probability with its limits stated in plain language (the in-domain F1, the out-of-domain false-positive risk, and the instruction to treat the score as a reason to talk, never as proof); it lists the SHAP-validated drivers of decisions in lecturer-readable terms; it presents each extracted claim quoted to its exact source sentences; it re-tags every question with the trained Bloom classifier, marking levels the classifier is weak on as advisory; and it closes with a suggested three-level rubric for reading the student's answers. The guide renders to Markdown, Word, and PDF (`outputs/verification_guides/3108a_ai_guide.pdf` is the worked example, generated end to end with live models). One design note carried through the document: the framing on the first page states that the guide is evidence for a conversation, not an accusation, which is the position the fairness results of Chapter 6 make necessary.

## 7.8 Fine-tuning the local backend: a result that looked too good

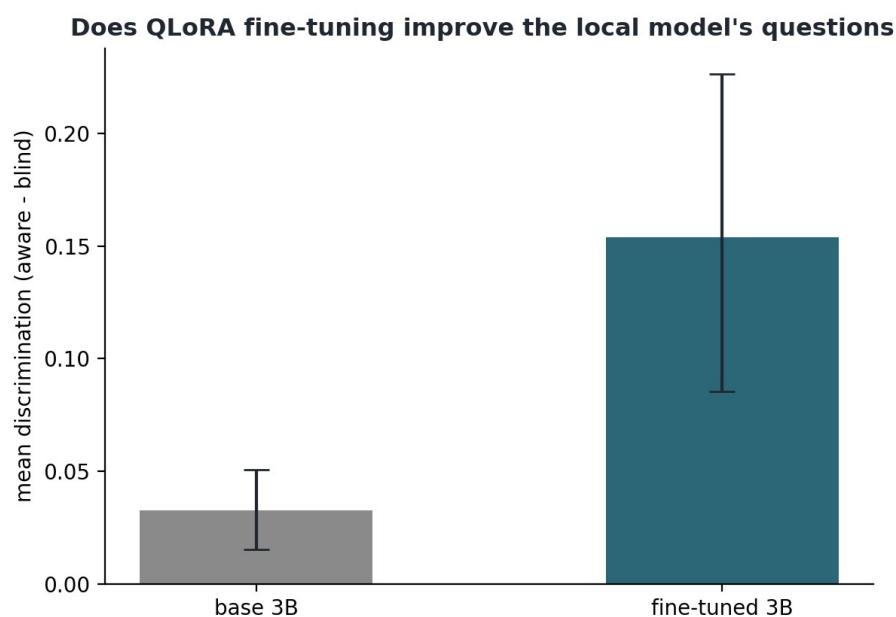
Section 7.4 left one question open for my supervisor: whether Backend B, the open-source side of the core comparison, should be the full Llama 3 8B with QLoRA or a smaller model that fits the laptop. The fit probes settled the practical half of it. In 4-bit with a LoRA adapter the 8B loads at 5.3 GB but only trains by spilling past the physical 8 GB into system memory, at 214 seconds per step for a 1024-token sequence, whereas Qwen2.5 3B trains inside VRAM at 2.8 seconds per step with about 3 GB of headroom (`outputs/qlora_fit_probe.json`). On the strength of those probes my supervisor signed off on Qwen2.5 3B as Backend B on 3 July 2026, so the fine-tuning experiment runs on the 3B.

I fine-tuned Qwen2.5-3B-Instruct with QLoRA (`src/question_gen/finetune_qg.py`): 4-bit NF4 base, a LoRA adapter (rank 16) on the attention projections, gradient checkpointing and a paged 8-bit optimiser, batch 1 with gradient accumulation of 8, two epochs over 2,600 passage-to-question pairs from EduQG. The prompt tokens are masked in the loss, so the

model learns only to produce the question. Training finished in under an hour on the 4060 at a final loss of 1.42, and the adapter is saved to `models/qg_finetune_qwen3b/`.

The evaluation is built to isolate the fine-tuning effect and nothing else (`src/question_gen/eval_finetune.py`). One fixed set of 18 claims was extracted once from six flagged essays. The base Qwen 3B and the fine-tuned Qwen 3B each wrote verification questions for those same claims, and both sets were scored by the same discrimination simulation used in Chapter 8. The only thing that changes between the two conditions is the question-writing model, so any difference is the adapter. The three phases never share the 8 GB: claims are extracted with the local model through Ollama, which is then unloaded, each transformer model generates in turn, and scoring loads the embedding model last.

On the raw metric the fine-tune looks like a triumph. The base model's questions score a mean discrimination of 0.033 (95 percent CI 0.015 to 0.051); the fine-tuned model's score 0.154 (95 percent CI 0.085 to 0.227), a Mann-Whitney  $p$  of 0.007 and confidence intervals that do not overlap (Figure 7.3). Taken at face value that is roughly a five-fold gain, and the same 0.15 shows up again at essay scale in Section 8.8. I nearly wrote it up as the headline result of the whole project.



*Figure 7.3: The QLoRA fine-tune of Qwen2.5 3B against its own base model on the same 18 claims, mean discrimination with 95 percent bootstrap intervals. The raw score rises from 0.033 to 0.154, but the quality audit in Section 8.9 shows this rise to be an artifact of degenerate output rather than better questions.*

Reading the actual questions stopped that. The fine-tuned model had overfit the format of its training data. EduQG is largely a multiple-choice corpus, and the adapter learned to emit multiple-choice stems: about 95 percent of its questions are strings like "Which of the following is correct?" or "Which of the following is not a reason why Peugeot is successful?", with no options ever supplied, and a handful are raw JSON fragments leaking from the prompt. These are not verification questions at all. A student could not answer them, and a lecturer could not use them. The full audit across all four models is in Section 8.9, where the base 3B, the Llama 8B and the commercial model each produce zero degenerate questions and the fine-tuned model produces almost only degenerate ones.

Worse, those empty stems are precisely what drove the score up. The literal string "Which of the following is correct?" scores a mean discrimination of 0.44, higher than any real question from any model. It does so because a contentless question defeats the simulation: the source-aware and the source-blind answerer both receive a stem with nothing to answer, their responses diverge more or less at random, and the aware-minus-blind gap the metric reports reflects that randomness, not any property a real student would need the essay to satisfy. So the 0.154 measures how empty the fine-tuned model's questions are, not how good they are. The honest conclusion is the opposite of the raw number: the QLoRA fine-tune on EduQG failed as a question generator for this task.

I keep this in the dissertation rather than deleting it because the failure is informative twice over. First, it shows that the training data's format matters more than its size: two epochs on an overwhelmingly multiple-choice corpus was enough to collapse a capable instruct model into producing multiple-choice stems, so the fix is not more data but the right data, a question-generation set of open-ended questions (or the verification-style prompts themselves), which is the clear next experiment for Backend B. Second, it exposes a real weakness in the discrimination simulation: the measure assumes well-formed questions and can be gamed by degenerate ones, so it needs a well-formedness guard (a simple filter, or the Bloom classifier, rejecting non-questions) before its scores are trusted. That caution is exactly why the evaluation plan never rested on a single automatic metric, and Section 8.9 turns the episode into that methodological point.

## Chapter 8: Evaluating the questions (discrimination simulation)

### 8.1 The idea

A good verification question is one that a student who understands their own work can answer and a student who does not cannot. I evaluate that without humans or an LLM judge, using a discrimination simulation (`src/evaluation/discrimination_sim.py`). For each question, one model answers it WITH the source passage from the essay (the "context-aware" answer) and another answers it with the question ONLY (the "context-blind" answer). I then measure how close each answer is to the source, using embedding similarity, and take the gap, aware minus blind, as the discrimination score. A high gap means the question genuinely needs the source; a near-zero gap means it can be answered without it. Everything runs on the local model and local embeddings, so the evaluation needs no API key and no judge.

### 8.2 The result, which surprised me

I expected my claim-grounded questions to discriminate well. They did not. On the flagged essay, the claim-grounded questions scored a mean discrimination of about 0.05 (95% confidence interval [0.01, 0.10]), while a set of generic essay questions ("what is your main argument", "what evidence most supports your conclusion", "explain your reasoning in your own words") scored about 0.31 ([0.27, 0.36]). The intervals do not overlap, so the difference is real, and it is the opposite of what I assumed (Figure 8.1).

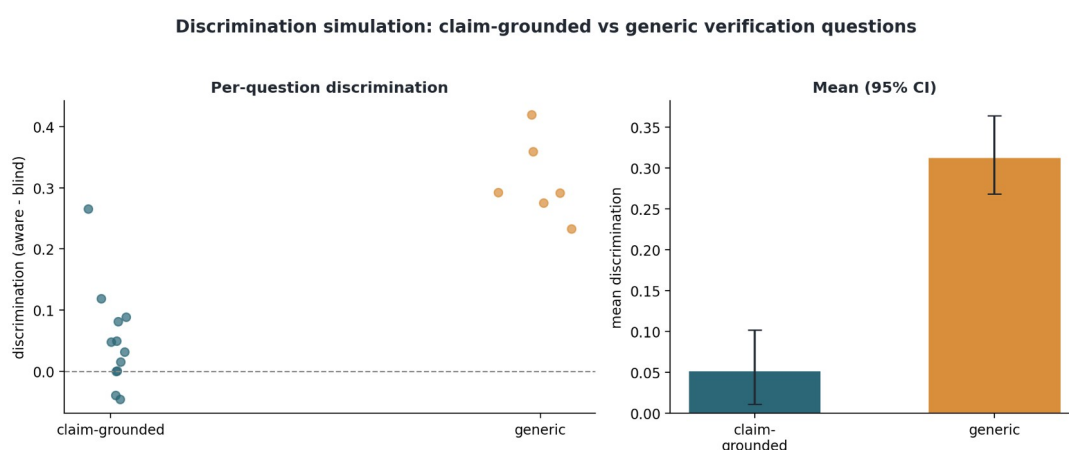


Figure 8.1: Discrimination simulation. Claim-grounded questions (left) cluster low; generic essay questions (right) discriminate much more. The gap is the source-aware minus source-blind answer similarity, with 95% confidence intervals.



### 8.3 Why, and what it means

The reason is the context-blind model. It is a large language model with broad world knowledge, so a question that names specific content, "what features make Shakespeare's dialogue intense", is answerable from general knowledge without ever reading this student's essay. The source barely changes the answer, so the discrimination is low. A generic question like "what is the main argument of your essay" is different: the blind model has no idea what this particular student argued, so its answer is vague, while the source-aware model answers about the actual essay. The source matters a lot, so the discrimination is high.

So the simulation is really measuring essay-specificity: does answering the question require this student's own essay, or only knowledge of the subject. That reframes what a good verification question is. It should force the student to reproduce their own argument, evidence, and choices, rather than recite facts about the topic that anyone knowledgeable could supply. When I rewrote the question-generation prompt to ask for the student's own reasoning and chosen evidence rather than general facts, the grounded score did improve, from about 0.03 to about 0.05, but it stayed well below the generic questions, because a specific question still tends to name the content and hand a knowledgeable answerer most of what it needs.

### 8.4 A caveat about the stand-in

The context-blind model is a strong stand-in, with far more world knowledge than a student who used AI without understanding the work. So a low discrimination score does not mean a question is useless in practice; it means the question is answerable by a knowledgeable person who never read this essay. The simulation is therefore a conservative, hard test of essay-specificity. A real student who did not understand their submission would do worse than the blind model on the grounded questions, so the true picture sits somewhere between the two lines in Figure 8.1.

### 8.5 What this changes

This is the first measured result for the question generation, and it is more useful as a finding than a score. It says the strongest verification questions are the ones that make the student retrieve and justify their own essay, its argument, the specific evidence they chose, and how their points connect, rather than questions that restate subject facts. The next iteration of the

generator will blend the claim grounding (so each question is tied to a real passage) with this retrieval-forcing style (so the answer cannot come from general knowledge). The remaining evaluation step is the supplementary LLM-as-judge rubric with cross-model agreement as a second view; the two steps promised here earlier, scaling the simulation across many essays and bringing in the commercial backend, are reported next.

## 8.6 Commercial versus local: the first scaled comparison

With the simulation in place, I ran the comparison at the centre of the research question. ATU could not provide API access or credits, so the commercial backend uses a free-tier Google Gemini API, plugged in behind the same interface as the local model; the local backend is the same Llama 3.1 8B used throughout. One disclosure on the commercial arm: free-tier quota limits forced a switch of Gemini endpoint partway through (four essays ran on `gemini-2.5-flash`, ten on `gemini-flash-latest`), so the commercial arm is one provider but not one fixed model snapshot. For each essay in a seeded random sample, each backend generated a Verification Interview Guide (three claims, grounded questions with sentence-level provenance), and every question from both backends was scored by the same local discrimination simulation, so what varies is which model wrote the questions.

A small pilot on the single flagged essay from Section 8.2 came first: there, both backends' grounded questions had discrimination intervals that included zero (local 0.032 with  $[-0.005, 0.083]$ ; commercial 0.022 with  $[-0.002, 0.048]$ ), too little signal to separate anything. The scaled run below is the real comparison.

Fifteen essays were sampled; fourteen produced a scored guide from both backends (one essay was excluded because claim extraction returned an empty guide; one commercial run was rate-limited for a day and completed on retry), giving each backend 126 questions. The pooled means are local 0.042 with a 95% interval of  $[0.030, 0.055]$  against commercial 0.078 with  $[0.058, 0.098]$ . Because the same essays sit under both backends, the fair test is paired: the commercial backend scores higher on 10 of the 14 essays, with a mean paired difference of 0.036, bootstrap interval  $[0.008, 0.067]$ , paired t-test  $p = 0.040$ , and Wilcoxon signed-rank  $p = 0.030$ . Unlike in the pilot, both pooled means sit clearly above zero, so at this scale the grounded questions do measurably need the source (Figure 8.2).

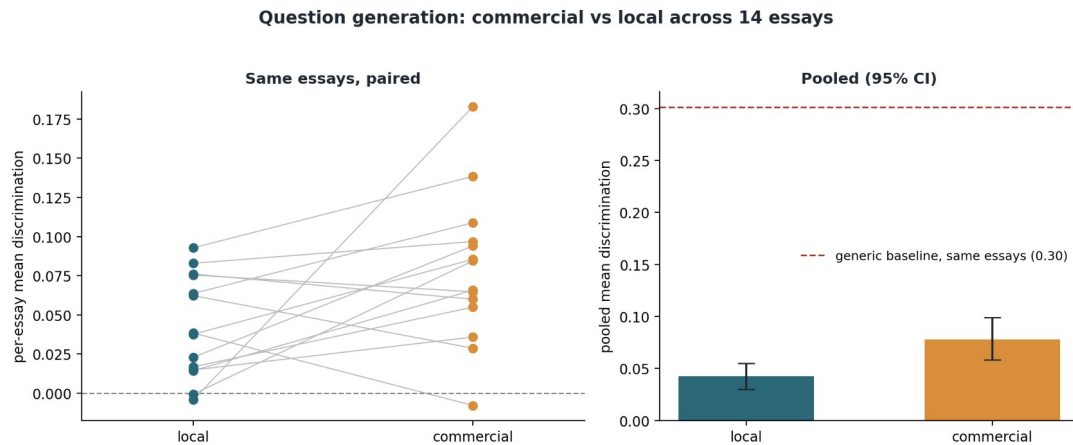


Figure 8.2: Commercial versus local question generation, balanced across the same 14 essays. Left: per-essay mean discrimination, paired. Right: pooled means with bootstrap 95% confidence intervals; the dashed line is the generic-question baseline.

The plain reading is that the commercial backend holds a small but statistically significant advantage on this measure, and that the local model remains competitive in absolute terms: the gap is about 0.04 against a generic-question baseline of 0.30 with a 95% interval of [0.28, 0.32], now re-measured on the same fourteen essays rather than taken from the pilot (84 generic questions; the pilot's single-essay value of 0.31 held up almost exactly), and it comes from a base model that has not yet been fine-tuned. Whether the planned QLoRA step closes that gap is exactly what the fine-tuning experiment will test. The sample-size trajectory is itself worth recording: nine essays showed no detectable difference, thirteen sat on the boundary ( $t p = 0.053$ ), and the full fourteen crossed it. Small-n snapshots of this comparison would have supported whichever story one preferred, which is why the dissertation reports the complete set with paired tests. The remaining caveats stand: the discrimination measure is conservative, both backends sit well below the generic-question baseline, and the commercial arm is one provider's free tier rather than a frontier model. Extending the comparison to more essays, more providers, and the fine-tuned local backend is mechanical from here, since the framework saves per essay and resumes.

## 8.7 LLM-as-judge: the first judge, anchored and found wanting

The supplementary evaluation puts an LLM judge over the questions with the four-dimension rubric (relevance, specificity, discrimination potential, cognitive appropriateness, each 1 to 5), and the plan has always been to validate judges rather than trust them. The first judge (the free-tier commercial model) has now rated all twelve grounded questions from the pilot guide

(src/evaluation/llm\_judge.py, resumable across quota interruptions; results in outputs/llm\_judge.json).

Two findings. First, a ceiling effect: the judge rates every question between 4.5 and 5.0, so its scores barely vary and mostly say that the questions look well-formed. Second, and the reason the anchoring exists: the judge's ratings show no meaningful correlation with the objective discrimination simulation on the same questions (Spearman rho of -0.14 for the judge's mean rating, -0.22 for its discrimination dimension, both far from significance at  $n = 12$ ). The judge is confident about exactly the questions the simulation shows a knowledgeable non-reader can answer, so judge scores alone would certify question quality that the objective measure contradicts.

The correlation's own history is a small- $n$  caution worth recording: at seven questions it looked strongly negative (about -0.7), and it attenuated toward zero as the remaining questions were scored. The honest summary is not "the judge is anti-correlated" but "the judge's near-ceiling ratings carry no signal about measured discrimination", which is precisely why the evaluation plan anchors every judge to the simulation and requires cross-model agreement before judge scores count. Completing that requires the second and third judges, which is a budget decision recorded in the supervision notes.

## 8.8 Like-for-like: the four writers on one fixed claim set

Section 8.6 compared the backends but let each one run its whole pipeline, so the model that chose the claims differed between arms as well as the model that wrote the questions. That leaves an ambiguity: the commercial edge there could come from writing better questions, or simply from Gemini selecting claims that happen to discriminate more. This section removes the ambiguity. One claim set is fixed per essay, extracted once with the neutral local extractor (Llama 3.1 8B), and four question writers answer the same claims: Llama 3.1 8B, free-tier Gemini, the base Qwen2.5 3B, and the QLoRA-fine-tuned Qwen2.5 3B. Every question is scored by the same discrimination simulation, so the only variable across the four arms is the model that writes the question (src/evaluation/likeforlike\_4way.py, outputs/likeforlike\_4way.json). The 14 essays are the balanced set from Section 8.6.

On the discrimination score the ordering looks dramatic (Figure 8.3). The fine-tuned 3B reaches a pooled mean of 0.153 (95% CI [0.106, 0.202]), far above the base 3B at 0.041 [0.028,

0.055], Llama 8B at 0.031 [0.016, 0.045], and commercial Gemini at 0.017 [-0.001, 0.036]. Its paired lead is 0.166 over commercial ( $p = 0.001$ ) and 0.123 over its own base ( $p < 0.001$ ), and it reproduces the 0.154 from Section 7.8. Read at face value, the locally fine-tuned open model does not merely match the commercial one, it buries it.

That reading does not survive looking at the questions, and Section 8.9 is given over to why: about 95 percent of the fine-tuned model's questions are degenerate multiple-choice stems that game the simulation, so the 0.153 measures how empty they are, not how good. The fine-tuned arm is therefore set aside for the interpretation here, and the honest comparison is among the three models whose questions are well-formed.

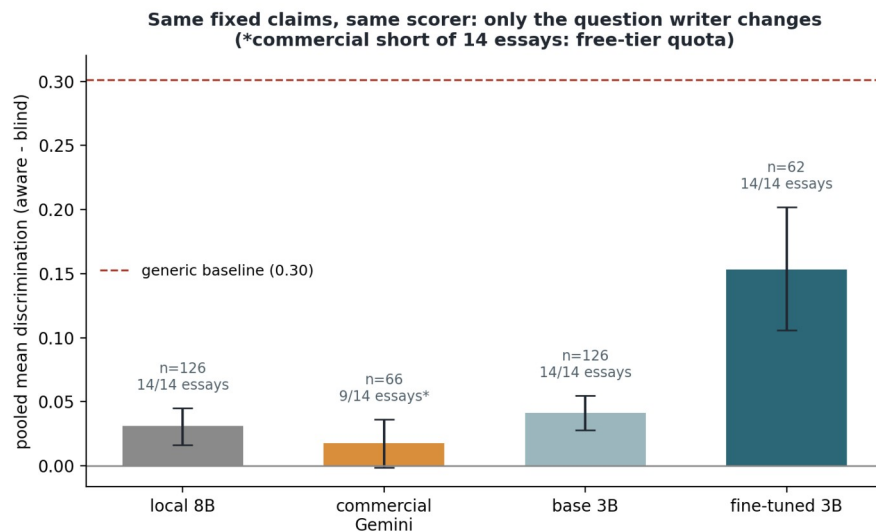


Figure 8.3: The four question writers on one fixed claim set per essay, same scorer, pooled mean discrimination with 95% bootstrap intervals, each bar labelled with its question count and essay coverage. The fine-tuned 3B posts the highest raw score, but Section 8.9 shows that score to be an artifact of degenerate questions; the meaningful comparison is among the other three arms.

Among those three the picture is modest and close. On the fixed claims the two open models edge the commercial one: base Qwen 3B at 0.041 and Llama 8B at 0.031 against Gemini at 0.017, though all three sit low and the commercial interval touches zero. Two things frame even this. The commercial arm is short, because free-tier quota ran out mid-run, so Gemini produced scored questions for only 9 of the 14 essays (66 questions); that comparison should be read as provisional until the arm is refilled to fourteen, which the framework resumes mechanically. And there is a real shift from Section 8.6. There, with each backend choosing its own claims, Gemini beat Llama 8B (0.078 to 0.042); here, on fixed claims, that advantage disappears (paired difference -0.016,  $p = 0.20$ , Gemini higher on only three of nine essays). The plausible reading is that part of the commercial edge in Section 8.6 came from Gemini

selecting easier-to-discriminate claims rather than from writing stronger questions on the same ones, but with nine paired essays that is a hypothesis for the full set, not a settled result. The firm conclusion of this section is narrower than the one I first drafted: on identical claims the small open models are competitive with free-tier commercial Gemini on this measure, while the second half of the research question, whether a fine-tuned local model can beat a commercial one, stays open, because the fine-tune that was supposed to answer it produced unusable questions.

## 8.9 The fine-tuned model games the metric: a quality audit

The previous section leaned on a number without reading what it scored. This section does the reading, because the gap between the two is the most important methodological lesson in the project (`src/evaluation/qg_quality_audit.py`, `outputs/qg_quality_audit.json`). Every question from all four arms was checked with a transparent rule for degeneracy: multiple-choice stems (which never carry options here), raw JSON fragments leaking from the prompt, and contentless "which is correct" forms.

The result is stark (Figure 8.4). The Llama 8B, base 3B and commercial arms produce no degenerate questions at all. The fine-tuned 3B produces almost nothing else: 59 of its 62 questions, just over 95 percent, are stems like "Which of the following is correct?" or "Which of the following is not a reason why Peugeot is successful?", with no options supplied. The fine-tune, two epochs on the largely multiple-choice EduQG corpus, had overfit the format of the training data and forgotten how to write an open question. None of these are usable in a verification interview.

The second panel is the part that matters for the evaluation method. Those empty stems do not merely survive the discrimination simulation, they win it. The single literal string "Which of the following is correct?", scored eleven times across the run, averages a discrimination of 0.44, higher than the mean of every model's real questions. The reason is structural: a contentless question gives the source-aware answerer and the source-blind answerer nothing to latch onto, their answers wander apart for reasons unrelated to the source, and the aware-minus-blind gap the metric rewards is large. The discrimination simulation, in other words, silently assumes the questions it scores are well-formed, and a degenerate generator can exploit that assumption to post an arbitrarily high score.

### Why the fine-tuned model's discrimination score is an artifact

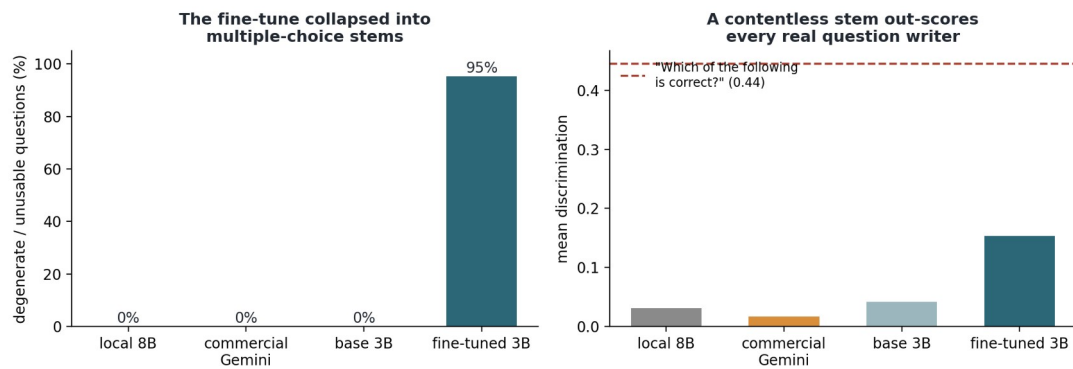


Figure 8.4: Auditing question quality behind the discrimination score. Left: the share of degenerate, unusable questions per model. Right: the mean discrimination of each model's questions against the score of the contentless stem "Which of the following is correct?" (dashed line), which out-scores every real question writer.

Two conclusions follow, and both are more useful than the false headline they replace. For Backend B, the lesson is that the training data's format dominates: fixing the fine-tune means not more data but the right data, an open-ended question-generation set (SQuAD-style questions, or the pipeline's own verification prompts distilled into training pairs) rather than a multiple-choice corpus, and that is the next experiment, carried out in Section 8.10. For the evaluation, the lesson is that the judge-free discrimination simulation, valuable as it is for well-formed questions, needs a well-formedness gate in front of it (the Bloom classifier already rejects non-questions, or a one-line filter can), and that no automatic score should be read without inspecting the text behind it. That is precisely the discipline the evaluation plan was built on, which is why this failure was caught here rather than in a viva.

## 8.10 Fixing the fine-tune: v2 on open-ended data

Section 8.9 made a testable prediction: if the degeneracy was caused by EduQG's multiple-choice format, then changing only the training data to open-ended questions should fix it. So I re-ran the fine-tune with everything held identical except the data, swapping EduQG for SQuAD, whose questions are open-ended and passage-grounded (`src/question_gen/finetune_qg_v2.py`, adapter in `models/qg_finnetune_qwen3b_v2/`). The evaluation repeats the Section 7.8 design on the same 18 claims, base against the new adapter, and it audits degeneracy before reading any score (`src/question_gen/eval_qg_v2.py`, `outputs/qg_v2_eval.json`).

The prediction holds, cleanly. The v2 model's questions are 0 percent degenerate against v1's 95, so the format of the training data was indeed the cause, not fine-tuning as such (Figure



8.5). And this time the discrimination gain is real rather than an artifact, because the questions are well-formed. On the same claims, v2 scores a mean discrimination of 0.102 (95% CI [0.060, 0.144]) against the base model's 0.027 ([0.015, 0.040]), a Mann-Whitney  $p$  of 0.0003. The contrast with v1 is the whole point: v1's higher 0.154 was empty multiple-choice stems gaming the metric, while v2's lower 0.102 is genuine questions that genuinely need the source more than the base model's do.

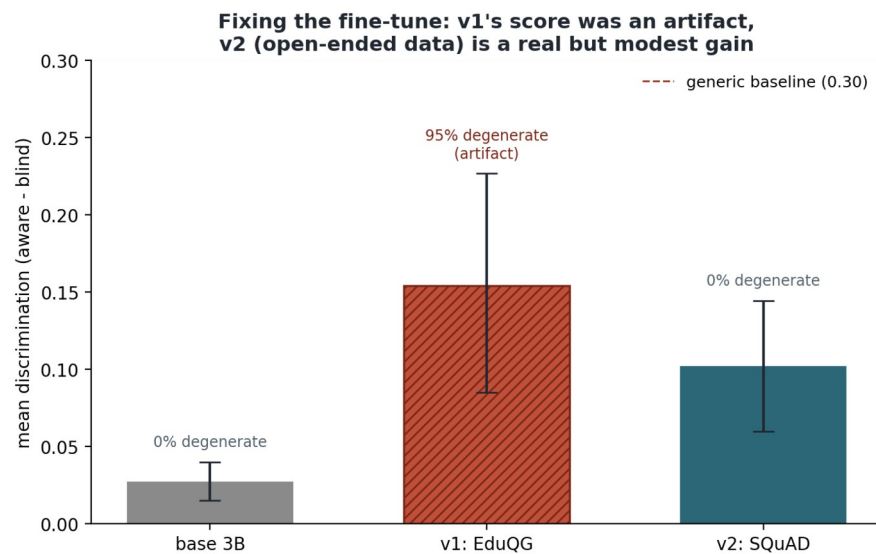


Figure 8.5: The fine-tune, corrected. Base 3B, the v1 EduQG adapter (hatched: its score is an artifact of 95 percent degenerate questions), and the v2 SQuAD adapter, mean discrimination on the same 18 claims with 95 percent intervals and each bar's degeneracy rate. v2 is a real gain over base ( $p = 0.0003$ ) on well-formed questions, but modest and still below the generic-question baseline.

I hold the reading to what was measured, because v2 is a partial success, not a triumph. Its 0.102 is well under the generic-question baseline of about 0.30, and reading its questions shows why: SQuAD trains a factual style ("What is the purpose of judicial review?"), and factual questions are partly answerable from general knowledge, which is exactly the property Section 8.3 showed the simulation penalises. Two smaller points round out the honesty. v2 needed a cleaned output parser, because the adapter sometimes appended JSON fragments after the question that an earlier extractor would have kept; the parser now takes the first well-formed question and drops the rest. And the degeneracy audit ran before the score, as the standing rule now requires. The overall arc is the useful result: fine-tuning a small local model does help once the training data has the right form, but SQuAD's factual questions are not the ideal signal for verification, so the genuinely next step is a training set of reasoning-demanding questions, distilled from the pipeline's own verification prompts, which would test whether the local backend can be pushed from this modest 0.102 toward



the commercial models' range on well-formed questions. That is where Backend B stands: a diagnosed and half-fixed component with a clear path, reported honestly rather than as the false win of Section 7.8's first draft.

## References

- Anderson, L. W. and Krathwohl, D. R. (2001). A Taxonomy for Learning, Teaching, and Assessing: A Revision of Bloom's Taxonomy of Educational Objectives. Longman, New York.
- DeYoung, J., Jain, S., Rajani, N. F., Lehman, E., Xiong, C., Socher, R. and Wallace, B. C. (2020). ERASER: A Benchmark to Evaluate Rationalized NLP Models. In Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics (ACL 2020), pp. 4443 to 4458.
- Guo, S., Liao, X., Li, C. and Chua, T.-S. (2024). A Survey on Neural Question Generation: Methods, Applications, and Prospects. In Proceedings of the 33rd International Joint Conference on Artificial Intelligence (IJCAI 2024). arXiv:2402.18267.
- Hadifar, A., Bitew, S. K., Deleu, J., Develder, C. and Demeester, T. (2022). EduQG: A Multi-format Multiple Choice Dataset for the Educational Domain. IEEE Access. arXiv:2210.06104.
- Hayes, A. F. and Krippendorff, K. (2007). Answering the Call for a Standard Reliability Measure for Coding Data. Communication Methods and Measures, 1(1), pp. 77 to 89.
- He, P., Liu, X., Gao, J. and Chen, W. (2021). DeBERTa: Decoding-enhanced BERT with Disentangled Attention. In Proceedings of the 9th International Conference on Learning Representations (ICLR 2021).
- Jain, S. and Wallace, B. C. (2019). Attention is not Explanation. In Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics (NAACL 2019), pp. 3543 to 3556.
- Jin, Y., Yan, L., Echeverria, V., Gasevic, D. and Martinez-Maldonado, R. (2024). Generative AI in Higher Education: A Global Perspective of Institutional Adoption Policies and Guidelines. arXiv:2405.11800.
- Krishna, K., Song, Y., Karpinska, M., Wieting, J. and Iyyer, M. (2023). Paraphrasing Evades Detectors of AI-Generated Text, but Retrieval is an Effective Defense. In Advances in Neural Information Processing Systems 36 (NeurIPS 2023). arXiv:2303.13408.
- Kumar, S., Gulwani, R. and Singh, P. (2025). Automated Analysis of Learning Outcomes and Exam Questions Based on Bloom's Taxonomy. arXiv:2511.10903.

Liang, W., Yuksekgonul, M., Mao, Y., Wu, E. and Zou, J. (2023). GPT Detectors Are Biased Against Non-Native English Writers. *Patterns*, 4(7), 100779. arXiv:2304.02819.

Liu, Y., Ott, M., Goyal, N., Du, J., Joshi, M., Chen, D., Levy, O., Lewis, M., Zettlemoyer, L. and Stoyanov, V. (2019). RoBERTa: A Robustly Optimized BERT Pretraining Approach. arXiv:1907.11692.

Lundberg, S. M. and Lee, S.-I. (2017). A Unified Approach to Interpreting Model Predictions. In *Advances in Neural Information Processing Systems 30 (NeurIPS 2017)*, pp. 4766 to 4777.

Mindner, L., Schlippe, T. and Schaaff, K. (2023). Classification of Human- and AI-Generated Texts: Investigating Features for ChatGPT. arXiv:2308.05341.

Mitchell, E., Lee, Y., Khazatsky, A., Manning, C. D. and Finn, C. (2023). DetectGPT: Zero-Shot Machine-Generated Text Detection Using Probability Curvature. In *Proceedings of the 40th International Conference on Machine Learning (ICML 2023)*, PMLR 202, pp. 24950 to 24962.

Mohammadshahi, A., Scialom, T., Yazdani, M., Yanki, P., Fan, A., Henderson, J. and Saeidi, M. (2023). RQUGE: Reference-Free Metric for Evaluating Question Generation by Answering the Question. In *Findings of the Association for Computational Linguistics: ACL 2023*, pp. 6845 to 6867. arXiv:2211.01482.

Oketch, K., Lalor, J. P., Yang, Y. and Abbasi, A. (2025). Bridging the LLM Accessibility Divide? Performance, Fairness, and Cost of Closed versus Open LLMs for Automated Essay Scoring. arXiv:2503.11827.

Pietron, M., Olszowski, R. and Gomulka, J. (2024). Efficient Argument Classification with Compact Language Models and ChatGPT-4 Refinements. arXiv:2403.15473.

Stab, C. and Gurevych, I. (2017). Parsing Argumentation Structures in Persuasive Essays. *Computational Linguistics*, 43(3), pp. 619 to 659.

Sundararajan, M., Taly, A. and Yan, Q. (2017). Axiomatic Attribution for Deep Networks. In *Proceedings of the 34th International Conference on Machine Learning (ICML 2017)*, PMLR 70, pp. 3319 to 3328.

Wang, Y., Mansurov, J., Ivanov, P., Su, J., Shelmanov, A., Tsvigun, A., et al. (2023). M4: Multi-Generator, Multi-Domain, and Multi-Lingual Black-Box Machine-Generated Text Detection. arXiv:2305.14902.

Wang, Y., Mansurov, J., Ivanov, P., Su, J., Shelmanov, A., Tsvigun, A., Mohammed Afzal, O., Mahmoud, T., Puccetti, G., Arnold, T., et al. (2024). SemEval-2024 Task 8: Multidomain, Multimodel and Multilingual Machine-Generated Text Detection. In Proceedings of the 18th International Workshop on Semantic Evaluation (SemEval-2024), pp. 2057 to 2079. arXiv:2404.14183.

Wu, J., Yang, S., Zhan, R., Yuan, Y., Chao, L. S. and Wong, D. F. (2025). A Survey on LLM-Generated Text Detection: Necessity, Methods, and Future Directions. Computational Linguistics, 51(1), pp. 275 to 338. arXiv:2310.14724.

Zheng, L., Chiang, W.-L., Sheng, Y., Zhuang, S., Wu, Z., Zhuang, Y., Lin, Z., Li, Z., Li, D., Xing, E. P., Zhang, H., Gonzalez, J. E. and Stoica, I. (2023). Judging LLM-as-a-Judge with MT-Bench and Chatbot Arena. In Advances in Neural Information Processing Systems 36 (NeurIPS 2023), Datasets and Benchmarks Track. arXiv:2306.05685.